Arrays

Array ko int main me define karo to garbage value se initialize hota hai, Agar globally karo to 0 se initialize hota hai.
10 power 6 is maximum length of array we can define (int main k andar)
10 power 7 is maximum length of array we can define (global declaration)

1. Find the Largest element in an array

⭐ APPROACH 1 → Using sort()

```
#include<bits/stdc++.h>
using namespace std;

int sortArr(vector<int>& arr) {
    sort(arr.begin(),arr.end());
    return arr[arr.size()-1];
}

int main() {
    vector<int> arr1 = {2,5,1,3,0};
    vector<int> arr2 = {8,10,5,7,9};

    cout<<"The Largest element in the array is: "<<sortArr(arr1)<<endl;
    cout<<"The Largest element in the array is: "<<sortArr(arr2);

    return 0;
}
```

Complexity Analysis

Time Complexity: O(N*log(N))

Space Complexity: O(1)

⭐ APPROACH 2 → Linear Scan (BEST)

```
#include <bits/stdc++.h>

using namespace std;
int findLargestElement(int arr[], int n) {
```

```
  int max = arr[0];
  for (int i = 0; i < n; i++) {
    if (max < arr[i]) {
      max = arr[i];
    }
  }
  return max;
}
int main() {
  int arr1[] = {2,5,1,3,0};
  int n = 5;
  int max = findLargestElement(arr1, n);
  cout << "The largest element in the array is: " << max << endl;

  int arr2[] =  {8,10,5,7,9};
  n = 5;
  max = findLargestElement(arr2, n);
  cout << "The largest element in the array is: " << max<<endl;
  return 0;
}
```

Complexity Analysis

Time Complexity: O(N)

Space Complexity: O(1)

2. Find Second Smallest and Second Largest Element in an array

⭐ APPROACH 1 → Sorting

```
#include<bits/stdc++.h>
using namespace std;
void getElements(int arr[],int n)
{
```

```
    if(n==0 || n==1)
        cout<<-1<<" "<<-1<<endl;  // edge case when only one element is present in array
    sort(arr,arr+n);
    int small=arr[1];
    int large=arr[n-2];
    cout<<"Second smallest is "<<small<<endl;
    cout<<"Second largest is "<<large<<endl;
}
int main()
{
    int arr[]={1,2,4,6,7,5};
    int n=sizeof(arr)/sizeof(arr[0]);
    getElements(arr,n);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(NlogN), For sorting the array

Space Complexity: O(1)

- Isme 1 aur case hai jaise 1 2 4 5 7 7, to isme 7 2nd largest nhi hai, hume n-2 se back loop run karni padegi 2nd largest k liye.

⭐ APPROACH 2 → Two Pass Approach (Better)

```
#include<bits/stdc++.h>
using namespace std;
void getElements(int arr[],int n)
{
    if(n==0 || n==1)
        cout<<-1<<" "<<-1<<endl;  // edge case when only one element is present in array
    int small=INT_MAX,second_small=INT_MAX;
    int large=INT_MIN,second_large=INT_MIN;
    int i;
    for(i=0;i<n;i++)
    {
        small=min(small,arr[i]);
        large=max(large,arr[i]);
    }
    for(i=0;i<n;i++)
    {
```

```
        if(arr[i]<second_small && arr[i]!=small)
            second_small=arr[i];
        if(arr[i]>second_large && arr[i]!=large)
            second_large=arr[i];
    }

    cout<<"Second smallest is "<<second_small<<endl;
    cout<<"Second largest is "<<second_large<<endl;
}
int main()
{
    int arr[]={1,2,4,6,7,5};
    int n=sizeof(arr)/sizeof(arr[0]);
    getElements(arr,n);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N), We do two linear traversals in our array

Space Complexity: O(1)

⭐ APPROACH 3 → Optimal (Single Pass)

```
#include<bits/stdc++.h>
using namespace std;
int secondSmallest(int arr[],int n)
{
    if(n<2)
        return -1;
    int small = INT_MAX;
    int second_small = INT_MAX;
    int i;
    for(i = 0; i < n; i++)
    {
      if(arr[i] < small)
      {
        second_small = small;
        small = arr[i];
      }
```

```
      else if(arr[i] < second_small && arr[i] != small)
      {
        second_small = arr[i];
      }
    }
   return second_small;
}
int secondLargest(int arr[],int n)
{
          if(n<2)
          return -1;
    int large=INT_MIN,second_large=INT_MIN;
    int i;
    for (i = 0; i < n; i++)
    {
        if (arr[i] > large)
        {
            second_large = large;
            large = arr[i];
        }

        else if (arr[i] > second_large && arr[i] != large)
        {
            second_large = arr[i];
        }
    }
    return second_large;
}

int main() {
    int arr[]={1,2,4,7,7,5};
    int n=sizeof(arr)/sizeof(arr[0]);
       int sS=secondSmallest(arr,n);
       int sL=secondLargest(arr,n);
    cout<<"Second smallest is "<<sS<<endl;
    cout<<"Second largest is "<<sL<<endl;
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N), Single-pass solution
Space Complexity: O(1)

3. Check if an Array is Sorted

🚫 Approach 1: Brute Force

```
#include <bits/stdc++.h>

using namespace std;

bool isSorted(int arr[], int n) {
  for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
      if (arr[j] < arr[i])
        return false;
    }
  }

  return true;
}

int main() {

  int arr[] = {1, 2, 3, 4, 5}, n = 5;
  bool ans = isSorted(arr, n);
  if (ans) cout << "True" << endl;
  else cout << "False" << endl;
  return 0;
}
```

Complexity Analysis

Time Complexity: O(N^2)

Space Complexity: O(1)

🌟 Approach 2: Optimal (O(N))

```
#include<bits/stdc++.h>

using namespace std;

bool isSorted(int arr[], int n) {
  for (int i = 1; i < n; i++) {
    if (arr[i] < arr[i - 1])
      return false;
  }

  return true;
}

int main() {

  int arr[] = {1, 2, 3, 4, 5}, n = 5;

  printf("%s", isSorted(arr, n) ? "True" : "False");

}
```

Complexity Analysis

Time Complexity: O(N)

Space Complexity: O(1)

4. Remove Duplicates in-place from Sorted Array

Input: 0 0 1 1 1 2 2 3 3 4
Output Array: 0 1 2 3 4 …aage kuch bhi ho
k = 5

❌ APPROACH 1: Using `unordered_set`

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int removeDuplicates(vector<int>& nums) {
        unordered_set<int> seen;
        int index = 0;
        for (int num : nums) {
            // If num is not in seen, it's unique
            if (seen.find(num) == seen.end()) {
                // Add this num to the set of seen numbers
                seen.insert(num);

                // Overwrite nums[index] with this unique num
                nums[index] = num;

                // Move index forward
                index++;
            }
        }
        // Return count of unique elements
        return index;
    }
};

int main() {
    vector<int> nums = {0,0,1,1,1,2,2,3,3,4};

    Solution sol;
    int k = sol.removeDuplicates(nums);
```

```
    cout << "k = " << k << "\nArray after removing duplicates: ";
    for (int i = 0; i < k; i++) {
        cout << nums[i] << " ";
    }
    cout << endl;
}
```

Complexity Analysis
Time Complexity: O(N), We traverse the entire array and insert elements into set.
Space Complexity: O(N), additional space used to store elements in set.

⭐ APPROACH 2 (Optimal): Two Pointer Technique

```
#include <bits/stdc++.h>
using namespace std;

// Class to hold the solution logic
class Solution {
public:
    // Function to remove duplicates from sorted array in-place
    int removeDuplicates(vector<int>& nums) {
        // If array is empty, return 0 directly
        if (nums.empty()) return 0;

        // Pointer for the position of last unique element
        int i = 0;

        // Traverse the array starting from the second element
        for (int j = 1; j < nums.size(); j++) {
            // If current element is different from last unique element
            if (nums[j] != nums[i]) {
                // Move pointer for unique element forward
                i++;
                // Place the new unique element at the next position
                nums[i] = nums[j];
            }
        }

        // i is index of last unique element, count = i + 1
        return i + 1;
```

```
    }
};

int main() {
    vector<int> nums = {0,0,1,1,1,2,2,3,3,4};

    Solution sol;
    int k = sol.removeDuplicates(nums);

    cout << "Unique count = " << k << "\n";
    cout << "Array after removing duplicates: ";
    for (int x = 0; x < k; x++) {
        cout << nums[x] << " ";
    }
    cout << endl;
}
```

Complexity Analysis
Time Complexity: O(N), We traverse the entire original array only once.
Space Complexity: O(1), constant additional space is used to check unique elements.

Dry run:
i=0 → nums[i]=0
j=1 → nums[1]=0 == nums[0] → skip duplicate
j=2 → nums[2]=1 != nums[0]
→ i = 1
→ nums[1] = 1
Array → 0 1 _ _ _ _ _

j=3 → nums[3]=1 == nums[1] → skip
j=4 → nums[4]=1 == nums[1] → skip

j=5 → nums[5]=2 != nums[1]
→ i = 2
→ nums[2] = 2

j=7 → nums[7]=3 != nums[2]
→ i=3
→ nums[3]=3

j=9 → nums[9]=4 != nums[3]
→ i=4
→ nums[4]=4

5. Left Rotate the Array by One

Input: [1, 2, 3, 4, 5]
Output: [2, 3, 4, 5, 1]

❌ Approach 1: Using Extra Array (Space = O(N))

```
#include<bits/stdc++.h>

using namespace std;
void solve(int arr[], int n) {
  int temp[n];
  for (int i = 1; i < n; i++) {
    temp[i - 1] = arr[i];
  }
  temp[n - 1] = arr[0];
  for (int i = 0; i < n; i++) {
    cout << temp[i] << " ";
  }
  cout << endl;
}
int main() {
  int n=5;

  int arr[]= {1,2,3,4,5};
  solve(arr, n);
}
```

Complexity Analysis

Time Complexity: O(n), as we iterate through the array only once.

Space Complexity: O(n) as we are using another array of size, same as the given array.

⭐ Approach 2 (Optimal): In-place Rotation (O(1) space)

```
#include<bits/stdc++.h>
```

```
using namespace std;
void solve(int arr[], int n) {
  int temp = arr[0]; // storing the first element of array in a variable
  for (int i = 0; i < n - 1; i++) {
    arr[i] = arr[i + 1];
  }
  arr[n - 1] = temp; // assigned the value of variable at the last index
  for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
  }

}
int main() {
  int n=5;

  int arr[]= {1,2,3,4,5};
  solve(arr, n);
}
```

Complexity Analysis

Time Complexity: O(n), as we iterate through the array only once.

Space Complexity: O(1) as no extra space is used

6. Rotate array by K elements

Right rotate by k: elements move to the right; items falling off the end appear at the start.
e.g. `arr=[1,2,3,4,5], k=2` → `[4,5,1,2,3]`

Left rotate by k: elements move to the left; first elements go to end.
e.g. `arr=[1,2,3,4,5], k=2` → `[3,4,5,1,2]`

Always normalize `k`: do `k = k % n`. If `k < 0` you can treat it as left rotate `(-k)` or convert to equivalent right rotate.

1) Method A — Temporary array (easy, uses O(k) extra space)
Idea: copy the k tail elements (for right-rotate) into temp, shift the rest right, then copy temp to beginning.

(Right Rotation)

```
#include <iostream>
using namespace std;
void Rotatetoright(int arr[], int n, int k)
{
  if (n == 0)
    return;
  k = k % n;
  if (k > n)
    return;
  int temp[k];
  for (int i = n - k; i < n; i++)
  {
    temp[i - n + k] = arr[i];
  }
  for (int i = n - k - 1; i >= 0; i--)
  {
    arr[i + k] = arr[i];
  }
  for (int i = 0; i < k; i++)
  {
    arr[i] = temp[i];
  }
}
int main()
{
  int n = 7;
  int arr[] = {1, 2, 3, 4, 5, 6, 7};
  int k = 2;
  Rotatetoright(arr, n, k);
  cout << "After Rotating the elements to right " << endl;
  for (int i = 0; i < n; i++)
  {
    cout << arr[i] << " ";
  }
  return 0;
}
```

Time Complexity: O(n)

Space Complexity: O(k) since k array element needs to be stored in temp array

(Left Rotate)

```
#include <iostream>
using namespace std;
void Rotatetoleft(int arr[], int n, int k)
{
  if (n == 0)
    return;
  k = k % n;
  if (k > n)
    return;
  int temp[k];
  for (int i = 0; i < k; i++)
  {
    temp[i] = arr[i];
  }
  for (int i = 0; i < n - k; i++)
  {
    arr[i] = arr[i + k];
  }
  for (int i = n - k; i < n; i++)
  {
    arr[i] = temp[i - n + k];
  }
}
int main()
{
  int n = 7;
  int arr[] = {1, 2, 3, 4, 5, 6, 7};
  int k = 2;
  Rotatetoleft(arr, n, k);
  cout << "After Rotating the elements to left " << endl;
  for (int i = 0; i < n; i++)
  {
    cout << arr[i] << " ";
```

```
  }
  return 0;
}
```

Time Complexity: O(n)

Space Complexity: O(k) since k array element needs to be stored in temp array

2) Method B — Reverse trick (recommended: O(n) time, O(1) space)

Idea (right rotate by k):

1. Reverse the whole array.
2. Reverse the first `k` elements.
3. Reverse the remaining `n-k` elements.
   That results in a right rotation.

   Equivalently some implement: reverse first `n-k`, then reverse last `k`, then reverse whole array — both yield same result.

Proof intuition

Reversing rearranges blocks; using three reverses moves the tail block to front while preserving internal order.

(Right Rotate)

```
#include <iostream>
using namespace std;
// Function to Reverse the array
void Reverse(int arr[], int start, int end)
{
  while (start <= end)
  {
    int temp = arr[start];
    arr[start] = arr[end];
    arr[end] = temp;
    start++;
```

```
    end--;
  }
}
// Function to Rotate k elements to right
void Rotateeletoright(int arr[], int n, int k)
{
  // Reverse first n-k elements
  Reverse(arr, 0, n - k - 1);
  // Reverse last k elements
  Reverse(arr, n - k, n - 1);
  // Reverse whole array
  Reverse(arr, 0, n - 1);
}
int main()
{
  int arr[] = {1, 2, 3, 4, 5, 6, 7};
  int n = 7;
  int k = 2;
  Rotateeletoright(arr, n, k);
  cout << "After Rotating the k elements to right ";
  for (int i = 0; i < n; i++)
    cout << arr[i] << " ";
  cout << endl;
  return 0;
}
```

Time Complexity - O(N) where N is the number of elements in an array

Space Complexity - O(1) since no extra space is required

(Left Rotate)

```
#include <iostream>
using namespace std;
// Function to Reverse the array
void Reverse(int arr[], int start, int end)
{
  while (start <= end)
  {
    int temp = arr[start];
    arr[start] = arr[end];
    arr[end] = temp;
    start++;
```

```
    end--;
  }
}
// Function to Rotate k elements to left
void Rotateeletoleft(int arr[], int n, int k)
{
  // Reverse first k elements
  Reverse(arr, 0, k - 1);
  // Reverse last n-k elements
  Reverse(arr, k, n - 1);
  // Reverse whole array
  Reverse(arr, 0, n - 1);
}
int main()
{
  int arr[] = {1, 2, 3, 4, 5, 6, 7};
  int n = 7;
  int k = 2;
  Rotateeletoleft(arr, n, k);
  cout << "After Rotating the k elements to left ";
  for (int i = 0; i < n; i++)
    cout << arr[i] << " ";
  cout << endl;
  return 0;
}
```

Time Complexity - O(N) where N is the number of elements in an array

Space Complexity - O(1) since no extra space is required

7. Move all Zeros to the end of the array

Input: 0 1 0 3 12
Output: 1 3 12 0 0

⭐ APPROACH 1 — Using Extra Array (Brute Force)

✔ Intuition

- Ek temp array banao (all 0s initially)
- Non-zero numbers ko left side me fill karo
- Baad me temp array ko original me copy kar do

```
#include <bits/stdc++.h>
using namespace std;

// Solution class
class Solution {
public:
    // Function to move all zeroes to end
    vector<int> moveZeroes(vector<int>& arr) {
        // Create temp array
        vector<int> temp(arr.size(), 0);

        // Pointer to fill temp
        int index = 0;

        // Traverse input array
        for (int i = 0; i < arr.size(); i++) {
            // If non-zero, add to temp
            if (arr[i] != 0) {
                temp[index] = arr[i];
                index++;
            }
        }

        // Copy back temp to original
        for (int i = 0; i < arr.size(); i++) {
            arr[i] = temp[i];
        }

        // Return updated array
        return arr;
    }
};

// Main function
int main() {
    vector<int> arr = {0, 1, 0, 3, 12};
    Solution sol;
```

```
    vector<int> result = sol.moveZeroes(arr);

    // Print result
    cout << "Array after moving zeroes: ";
    for (int num : result) {
        cout << num << " ";
    }
    cout << endl;
    return 0;
}
```

Time Complexity: O(N), we can move all zeroes to end in linear time.
Space Complexity: O(N), additional space used for temporary array.

⭐ APPROACH 2 — Optimal Two-Pointer / Swapping (O(1) space)

✔ Intuition (Simple)

first zero ka index chahiye:

Phir array left-to-right traverse karo:

Jab bhi non-zero mile
→ usse j-th index (first zero position) ke sath swap kar do
→ j++ (next zero ka index update ho jayega.

This keeps relative order preserved.

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    // Function to move zeroes to the end
    void moveZeroes(vector<int>& nums) {
        // Pointer to the first zero
        int j = -1;

        // Find the first zero
```

```
        for (int i = 0; i < nums.size(); i++) {
            if (nums[i] == 0) {
                j = i;
                break;
            }
        }

        // If no zero found, return
        if (j == -1) return;

        // Start from the next index of first zero
        for (int i = j + 1; i < nums.size(); i++) {
            // If current element is non-zero
            if (nums[i] != 0) {
                // Swap with nums[j]
                swap(nums[i], nums[j]);
                // Move j to next zero
                j++;
            }
        }
    }
};

int main() {
    Solution sol;
    vector<int> nums = {0, 1, 0, 3, 12};
    sol.moveZeroes(nums);

    // Print the result
    for (int num : nums) cout << num << " ";
    cout << endl;
    return 0;
}
```

Time Complexity: O(N), we can move all zeroes to end in linear time.
Space Complexity: O(1), constant additional space is used.

8. Linear Search in C

```
#include<stdio.h>

int search(int arr[],int n,int num)
{
   int i;
   for(i=0;i<n;i++)
   {
      if(arr[i]==num)
      return i;
   }
   return -1;
}
int main()
{
   int arr[]={1,2,3,4,5};
   int num = 4;
   int n = sizeof(arr)/sizeof(arr[0]);
   int val = search(arr,n,num);
   printf("%d",val);
}
```

Time Complexity: O(n), where n is the length of the array.

Space Complexity: O(1)

9. Union of Two Sorted Arrays

Approach A — `map` / `set` (easy, uses tree)

```
#include <bits/stdc++.h>

using namespace std;
```

```
vector < int > FindUnion(int arr1[], int arr2[], int n, int m) {
  map < int, int > freq;
  vector < int > Union;
  for (int i = 0; i < n; i++)
    freq[arr1[i]]++;
  for (int i = 0; i < m; i++)
    freq[arr2[i]]++;
  for (auto & it: freq)
    Union.push_back(it.first);
  return Union;
}

int main() {
  int n = 10, m = 7;
  int arr1[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
  int arr2[] = {2, 3, 4, 4, 5, 11, 12};
  vector < int > Union = FindUnion(arr1, arr2, n, m);
  cout << "Union of arr1 and arr2 is " << endl;
  for (auto & val: Union)
    cout << val << " ";
  return 0;
}
```

Time Compleixty : O((m+n)log(m+n)) . Inserting a key in map takes logN times, where N is no of elements in map. At max map can store m+n elements {when there are no common elements and elements in arr,arr2 are distntict}. So Inserting m+n th element takes log(m+n) time. Upon approximation across insertion of all elements in worst it would take O((m+n)log(m+n) time.

Using HashMap also takes the same time, On average insertion in unordered_map takes O(1) time but sorting the union vector takes O((m+n)log(m+n)) time. Because at max union vector can have m+n elements.

Space Complexity : O(m+n) {If Space of Union ArrayList is considered}

O(1) {If Space of union ArrayList is not considered}

Now using set

```
#include <bits/stdc++.h>

using namespace std;
```

```
vector < int > FindUnion(int arr1[], int arr2[], int n, int m) {
  set < int > s;
  vector < int > Union;
  for (int i = 0; i < n; i++)
    s.insert(arr1[i]);
  for (int i = 0; i < m; i++)
    s.insert(arr2[i]);
  for (auto & it: s)
    Union.push_back(it);
  return Union;
}

int main()

{
  int n = 10, m = 7;
  int arr1[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
  int arr2[] = {2, 3, 4, 4, 5, 11, 12};
  vector < int > Union = FindUnion(arr1, arr2, n, m);
  cout << "Union of arr1 and arr2 is  " << endl;
  for (auto & val: Union)
    cout << val << " ";
  return 0;
}
```

Time Compleixty : O((m+n)log(m+n)) . Inserting an element in a set takes logN time, where N is no of elements in the set. At max set can store m+n elements {when there are no common elements and elements in arr,arr2 are distntict}. So Inserting m+n th element takes log(m+n) time. Upon approximation across inserting all elements in worst, it would take O((m+n)log(m+n) time.

Using HashSet also takes the same time, On average insertion in unordered_set takes O(1) time but sorting the union vector takes O((m+n)log(m+n)) time. Because at max union vector can have m+n elements.

Space Complexity : O(m+n) {If Space of Union ArrayList is considered}

O(1) {If Space of union ArrayList is not considered}

Approach (BEST when inputs are sorted) — **Two-pointer merge**

```
#include <bits/stdc++.h>

using namespace std;

vector < int > FindUnion(int arr1[], int arr2[], int n, int m) {
  int i = 0, j = 0; // pointers
  vector < int > Union; // Uninon vector
  while (i < n && j < m) {
    if (arr1[i] <= arr2[j]) // Case 1 and 2
    {
      if (Union.size() == 0 || Union.back() != arr1[i])
        Union.push_back(arr1[i]);
      i++;
    } else // case 3
    {
      if (Union.size() == 0 || Union.back() != arr2[j])
        Union.push_back(arr2[j]);
      j++;
    }
  }
  while (i < n) // IF any element left in arr1
  {
    if (Union.back() != arr1[i])
      Union.push_back(arr1[i]);
    i++;
  }
  while (j < m) // If any elements left in arr2
  {
    if (Union.back() != arr2[j])
      Union.push_back(arr2[j]);
    j++;
  }
  return Union;
}

int main()

{
  int n = 10, m = 7;
  int arr1[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
  int arr2[] = {2, 3, 4, 4, 5, 11, 12};
```

```
  vector < int > Union = FindUnion(arr1, arr2, n, m);
  cout << "Union of arr1 and arr2 is  " << endl;
  for (auto & val: Union)
    cout << val << " ";
  return 0;
}
```

Time Complexity: O(m+n), Because at max i runs for n times and j runs for m times. When there are no common elements in arr1 and arr2 and all elements in arr1, arr2 are distinct.

Space Complexity : O(m+n) {If Space of Union ArrayList is considered}

O(1) {If Space of union ArrayList is not considered}

If arrays are already sorted → always use two-pointer.

If arrays unsorted and you need sorted union → either use set (simple) or sort both then two-pointer.

Intersection of Two Sorted Arrays

1) Brute Force (O(n1·n2)) — visited array ke saath

(A) Multiplicity (e.g., [1,2,2,3] & [2,2,2] → [2,2])

```
#include <bits/stdc++.h>
using namespace std;

// arr1 size n1, arr2 size n2 (SORTED assumed, but brute force works anyway)
vector<int> intersectionBruteMultiplicity(int arr1[], int n1, int arr2[], int n2) {
    vector<int> res;
    vector<int> visited(n2, 0); // mark matched indices in arr2

    for (int i = 0; i < n1; i++) {
        for (int j = 0; j < n2; j++) {
            if (!visited[j] && arr1[i] == arr2[j]) {
                res.push_back(arr1[i]);
                visited[j] = 1;           // this arr2[j] consumed
                break;                    // move to next arr1[i]
            }
            if (arr2[j] > arr1[i]) break; // small optimization if sorted
        }
```

```
    }
    return res;
}
```

- **Time:** O(n1·n2)
- **Space:** O(n2)

(B) Unique intersection (deduplicate result)

```
vector<int> intersectionBruteUnique(int arr1[], int n1, int arr2[], int n2) {
    vector<int> res;
    vector<int> visited(n2, 0);

    for (int i = 0; i < n1; i++) {
        // skip internal duplicates from arr1 to avoid re-adding same value
        if (i > 0 && arr1[i] == arr1[i-1]) continue;

        for (int j = 0; j < n2; j++) {
            if (!visited[j] && arr1[i] == arr2[j]) {
                res.push_back(arr1[i]);
                // we could mark visited[j]=1, but for unique result we can break directly
                break;
            }
            if (arr2[j] > arr1[i]) break;
        }
    }
    return res;
}
```

2) Two-Pointer (O(n1+n2)) — BEST for sorted arrays

(A) Unique intersection

```
#include <bits/stdc++.h>
using namespace std;

vector<int> intersectionTwoPointerUnique(int a[], int n1, int b[], int n2) {
```


Prepared by Vinay Kajla

Instagram: @Vinay.Kajla_

```
    int i = 0, j = 0;
    vector<int> res; res.reserve(min(n1, n2));

    while (i < n1 && j < n2) {
        if (a[i] == b[j]) {
            // push only once per value
            if (res.empty() || res.back() != a[i]) res.push_back(a[i]);
            // skip duplicates in both arrays
            int val = a[i];
            while (i < n1 && a[i] == val) i++;
            while (j < n2 && b[j] == val) j++;
        } else if (a[i] < b[j]) {
            i++;
        } else {
            j++;
        }
    }
    return res;
}
```

Time: O(n1 + n2)

Space: O(1) extra (output ke alawa)

(B) Multiplicity (counted intersection)

```
vector<int> intersectionTwoPointerMultiplicity(int a[], int n1, int b[], int n2) {
    int i = 0, j = 0;
    vector<int> res; res.reserve(min(n1, n2));

    while (i < n1 && j < n2) {
        if (a[i] == b[j]) {
            res.push_back(a[i]);
            i++; j++;
        } else if (a[i] < b[j]) {
            i++;
        } else {
            j++;
        }
    }
    return res;
}
```

Time: O(n1 + n2)

Space: O(1) extra

Arrays sorted → **Two-pointer** (fastest, cleanest)

Unique result chahiye → skip duplicates (as shown)

Multiplicity chahiye → equal pe push, both pointers ++ (no extra dedup)

10. Find the missing number in an array

1) Brute force (nested loops) — ❌

- For each `i` in `1..N`, linearly search in array

```
#include <bits/stdc++.h>
using namespace std;

int missingNumber(vector<int>&a, int N) {

    // Outer loop that runs from 1 to N:
    for (int i = 1; i <= N; i++) {

        // flag variable to check
        //if an element exists
        int flag = 0;

        //Search the element using linear search:
        for (int j = 0; j < N - 1; j++) {
            if (a[j] == i) {

                // i is present in the array:
                flag = 1;
```

```
            break;
          }
        }

        // check if the element is missing
        //i.e flag == 0:

        if (flag == 0) return i;
    }

    // The following line will never execute.
    // It is just to avoid warnings.
    return -1;
}

int main()
{
    int N = 5;
    vector<int> a = {1, 2, 4, 5};
    int ans = missingNumber(a, N);
    cout << "The missing number is: " << ans << endl;
    return 0;
}
```

Time Complexity: O(N2), where N = size of the array+1.
Reason: In the worst case i.e. if the missing number is N itself, the outer loop will run for N times, and for every single number the inner loop will also run for approximately N times. So, the total time complexity will be O(N2).

Space Complexity: O(1) as we are not using any extra space.

2) Hashing / freq array — ✅

```
#include <bits/stdc++.h>
using namespace std;

int missingNumber(vector<int>&a, int N) {

    int hash[N + 1] = {0}; //hash array
```

```
    // storing the frequencies:
    for (int i = 0; i < N - 1; i++)
        hash[a[i]]++;

    //checking the freqencies for numbers 1 to N:
    for (int i = 1; i <= N; i++) {
        if (hash[i] == 0) {
            return i;
        }
    }

    // The following line will never execute.
    // It is just to avoid warnings.
    return -1;
}

int main()
{
    int N = 5;
    vector<int> a = {1, 2, 4, 5};
    int ans = missingNumber(a, N);
    cout << "The missing number is: " << ans << endl;
    return 0;
}
```

Time Complexity: O(N) + O(N) ~ O(2*N), where N = size of the array+1.
Reason: For storing the frequencies in the hash array, the program takes O(N) time complexity and for checking the frequencies in the second step again O(N) is required. So, the total time complexity is O(N) + O(N).

Space Complexity: O(N), where N = size of the array+1. Here we are using an extra hash array of size N+1.

3) Sum formula — ✅ (watch overflow)

- Sum(1..N) − sum(array) = missing.

```
#include <bits/stdc++.h>
using namespace std;
```

```
int missingNumber(vector<int>&a, int N) {

    //Summation of first N numbers:
    int sum = (N * (N + 1)) / 2;

    //Summation of all array elements:
    int s2 = 0;
    for (int i = 0; i < N - 1; i++) {
        s2 += a[i];
    }

    int missingNum = sum - s2;
    return missingNum;
}

int main()
{
    int N = 5;
    vector<int> a = {1, 2, 4, 5};
    int ans = missingNumber(a, N);
    cout << "The missing number is: " << ans << endl;
    return 0;
}
```

Time Complexity: O(N), where N = size of array+1.
Reason: Here, we need only 1 loop to get the sum of the array elements. The loop runs for approx. N times. So, the time complexity is O(N).

Space Complexity: O(1) as we are not using any extra space.

4) XOR method — ⭐ Best (no overflow, O(1) space)
(1 ^ 2 ^ ... ^ N) ^ (a[0] ^ a[1] ^ ...) = missing

```
#include <bits/stdc++.h>
using namespace std;

int missingNumber(vector<int>&a, int N) {

    int xor1 = 0, xor2 = 0;
```

```
    for (int i = 0; i < N - 1; i++) {
        xor2 = xor2 ^ a[i]; // XOR of array elements
        xor1 = xor1 ^ (i + 1); //XOR up to [1...N-1]
    }
    xor1 = xor1 ^ N; //XOR up to [1...N]

    return (xor1 ^ xor2); // the missing number
}

int main()
{
    int N = 5;
    vector<int> a = {1, 2, 4, 5};
    int ans = missingNumber(a, N);
    cout << "The missing number is: " << ans << endl;
    return 0;
}
```

Time Complexity: O(N), where N = size of array+1.
Reason: Here, we need only 1 loop to calculate the XOR. The loop runs for approx. N times. So, the time complexity is O(N).

Space Complexity: O(1) as we are not using any extra space.

11. Count Maximum Consecutive One's in the array

Input: [1, 1, 0, 1, 1, 1]
Output: 3
Explanation: The maximum consecutive ones are the last three 1’s.

Intuition

Hum array me traverse karte hue ek counter `cnt` rakhenge,
jo batata hai abhi tak kitne consecutive 1’s chal rahe hain.

- Jab bhi `1` mile → `cnt++`
- Jab `0` mile → reset (`cnt = 0`)
- Har step par `maxi = max(maxi, cnt)` rakhenge

End me `maxi` hoga **maximum continuous 1s**.

```
#include <bits/stdc++.h>

using namespace std;
class Solution {
  public:
    int findMaxConsecutiveOnes(vector < int > & nums) {
      int cnt = 0;
      int maxi = 0;
      for (int i = 0; i < nums.size(); i++) {
        if (nums[i] == 1) {
          cnt++;
        } else {
          cnt = 0;
        }

        maxi = max(maxi, cnt);
      }
      return maxi;
    }
};

int main() {
  vector < int > nums = { 1, 1, 0, 1, 1, 1 };
  Solution obj;
  int ans = obj.findMaxConsecutiveOnes(nums);
  cout << "The maximum  consecutive 1's are " << ans;
  return 0;
}
```

Time Complexity: O(N) since the solution involves only a single pass.

Space Complexity: O(1) because no extra space is used.

12. Find the number that appears once, and the other numbers twice

Approach 1: Brute Force (Double Loop)

Idea: Har element ke liye count nikaalo, agar count = 1 → wahi answer.

```
#include <bits/stdc++.h>
using namespace std;

int getSingleElement(vector<int> &arr) {
    // Size of the array:
    int n = arr.size();

    //Run a loop for selecting elements:
    for (int i = 0; i < n; i++) {
        int num = arr[i]; // selected element
        int cnt = 0;

        //find the occurrence using linear search:
        for (int j = 0; j < n; j++) {
            if (arr[j] == num)
                cnt++;
        }

        // if the occurrence is 1 return ans:
        if (cnt == 1) return num;
    }

    //This line will never execute
    //if the array contains a single element.
    return -1;
}

int main()
{
    vector<int> arr = {4, 1, 2, 1, 2};
```

```
    int ans = getSingleElement(arr);
    cout << "The single element is: " << ans << endl;
    return 0;
}
```

Time Complexity: O(N2), where N = size of the given array.
Reason: For every element, we are performing a linear search to count its occurrence. The linear search takes O(N) time complexity. And there are N elements in the array. So, the total time complexity will be O(N2).

Space Complexity: O(1) as we are not using any extra space.

Approach 2: Hashing / Frequency Array

Idea: Store count of each number in a hash (array or map).

```
#include <bits/stdc++.h>
using namespace std;

int getSingleElement(vector<int> &arr) {

    //size of the array:
    int n = arr.size();

    // Find the maximum element:
    int maxi = arr[0];
    for (int i = 0; i < n; i++) {
        maxi = max(maxi, arr[i]);
    }

    // Declare hash array of size maxi+1
    // And hash the given array:
    vector<int> hash(maxi + 1, 0);
    for (int i = 0; i < n; i++) {
        hash[arr[i]]++;
    }

    //Find the single element and return the answer:
```

```
    for (int i = 0; i < n; i++) {
        if (hash[arr[i]] == 1)
            return arr[i];
    }

    //This line will never execute
    //if the array contains a single element.
    return -1;
}

int main()
{
    vector<int> arr = {4, 1, 2, 1, 2};
    int ans = getSingleElement(arr);
    cout << "The single element is: " << ans << endl;
    return 0;
}
```

Time Complexity: O(N)+O(N)+O(N), where N = size of the array
Reason: One O(N) is for finding the maximum, the second one is to hash the elements and the third one is to search the single element in the array.

Space Complexity: O(maxElement+1) where maxElement = the maximum element of the array.

Approach 3: Using `unordered_map`

Idea: Count frequency using hash map (dynamic key storage).

```
#include <bits/stdc++.h>
using namespace std;

int getSingleElement(vector<int> &arr) {

    //size of the array:
    int n = arr.size();

    // Declare the hashmap.
    // And hash the given array:
    map<int, int> mpp;
```

```
    for (int i = 0; i < n; i++) {
        mpp[arr[i]]++;
    }

    //Find the single element and return the answer:
    for (auto it : mpp) {
        if (it.second == 1)
            return it.first;
    }

    //This line will never execute
    //if the array contains a single element.
    return -1;
}

int main()
{
    vector<int> arr = {4, 1, 2, 1, 2};
    int ans = getSingleElement(arr);
    cout << "The single element is: " << ans << endl;
    return 0;
}
```

Time Complexity: O(N*logM) + O(M), where M = size of the map i.e. M = (N/2)+1. N = size of the array.
Reason: We are inserting N elements in the map data structure and insertion takes logM time(where M = size of the map). So this results in the first term O(N*logM). The second term is to iterate the map and search the single element. In Java, HashMap generally takes O(1) time complexity for insertion and search. In that case, the time complexity will be O(N) + O(M).

Note: The time complexity will be changed depending on which map data structure we are using. If we use unordered_map in C++, the time complexity will be O(N) for the best and average case instead of O(N*logM). But in the worst case(which rarely happens), it will be nearly O(N2).

Space Complexity: O(M) as we are using a map data structure. Here M = size of the map i.e. M = (N/2)+1.

Approach 4: XOR Trick (Optimal 🔥)

Concept:

- a ^ a = 0
- a ^ 0 = a
- XOR is associative and commutative

So if you XOR all numbers, the duplicate ones cancel out, leaving the unique one.

```
#include <bits/stdc++.h>
using namespace std;

int getSingleElement(vector<int> &arr) {
    //size of the array:
    int n = arr.size();

    // XOR all the elements:
    int xorr = 0;
    for (int i = 0; i < n; i++) {
        xorr = xorr ^ arr[i];
    }
    return xorr;
}

int main()
{
    vector<int> arr = {4, 1, 2, 1, 2};
    int ans = getSingleElement(arr);
    cout << "The single element is: " << ans << endl;
    return 0;
}
```

Time Complexity: O(N), where N = size of the array.
Reason: We are iterating the array only once.

Space Complexity: O(1) as we are not using any extra space.

13. Longest Subarray with given Sum K(Positives)

Input: arr = [2, 3, 5, 1, 9], K = 10
Output: 3
Explanation: The subarray [2, 3, 5] has sum = 10 and length = 3.

🐢 1. Brute Force — 3 Loops

💡 Idea

Try **every possible subarray**
 and for each subarray compute sum manually.

```
#include <bits/stdc++.h>
using namespace std;

int getLongestSubarray(vector<int>& a, long long k) {
    int n = a.size(); // size of the array.

    int len = 0;
    for (int i = 0; i < n; i++) { // starting index
        for (int j = i; j < n; j++) { // ending index
            // add all the elements of
            // subarray = a[i...j]:
            long long s = 0;
            for (int K = i; K <= j; K++) {
                s += a[K];
            }

            if (s == k)
                len = max(len, j - i + 1);
        }
    }
    return len;
}

int main()
{
    vector<int> a = {2, 3, 5, 1, 9};
```

```
    long long k = 10;
    int len = getLongestSubarray(a, k);
    cout << "The length of the longest subarray is: " << len << "\n";
    return 0;
}
```

Time Complexity: O(N3) approx., where N = size of the array.
Reason: We are using three nested loops, each running approximately N times.

Space Complexity: O(1) as we are not using any extra space.

🐇 2. Better Brute — 2 Loops

💡 Idea

Instead of recalculating sum in a third loop, keep a running sum inside the inner loop.

```
#include <bits/stdc++.h>
using namespace std;

int getLongestSubarray(vector<int>& a, long long k) {
    int n = a.size(); // size of the array.

    int len = 0;
    for (int i = 0; i < n; i++) { // starting index
        long long s = 0; // Sum variable
        for (int j = i; j < n; j++) { // ending index
            // add the current element to
            // the subarray a[i...j-1]:
            s += a[j];

            if (s == k)
                len = max(len, j - i + 1);
        }
    }
    return len;
}

int main()
{
```

```
    vector<int> a = {2, 3, 5, 1, 9};
    long long k = 10;
    int len = getLongestSubarray(a, k);
    cout << "The length of the longest subarray is: " << len << "\n";
    return 0;
}
```

Time Complexity: O(N2) approx., where N = size of the array.
Reason: We are using two nested loops, each running approximately N times.

Space Complexity: O(1) as we are not using any extra space.

3. Optimal (using Prefix Sum + Hash Map)

💡 Idea

If we maintain the prefix sum up to each index,
then:

```
sum(i...j) = prefix[j] - prefix[i-1]
```

If `prefix[j] - k` exists before, then there's a subarray with sum = k.

We store each prefix sum in a map with its first index.

```
#include <bits/stdc++.h>
using namespace std;

int getLongestSubarray(vector<int>& a, long long k) {
    int n = a.size(); // size of the array.

    map<long long, int> preSumMap;
    long long sum = 0;
    int maxLen = 0;
    for (int i = 0; i < n; i++) {
        //calculate the prefix sum till index i:
        sum += a[i];
```

```
        // if the sum = k, update the maxLen:
        if (sum == k) {
            maxLen = max(maxLen, i + 1);
        }

        // calculate the sum of remaining part i.e. x-k:
        long long rem = sum - k;

        //Calculate the length and update maxLen:
        if (preSumMap.find(rem) != preSumMap.end()) {
            int len = i - preSumMap[rem];
            maxLen = max(maxLen, len);
        }

        //Finally, update the map checking the conditions:
        if (preSumMap.find(sum) == preSumMap.end()) {
            preSumMap[sum] = i;
        }
    }

    return maxLen;
}

int main()
{
    vector<int> a = {2, 3, 5, 1, 9};
    long long k = 10;
    int len = getLongestSubarray(a, k);
    cout << "The length of the longest subarray is: " << len << "\n";
    return 0;
}
```

Time Complexity: O(N) or O(N*logN) depending on which map data structure we are using, where N = size of the array.
Reason: For example, if we are using an unordered_map data structure in C++ the time complexity will be O(N)(though in the worst case, unordered_map takes O(N) to find an element and the time complexity becomes O(N2)) but if we are using a map data structure, the time complexity will be O(N*logN). The least complexity will be O(N) as we are using a loop to traverse the array.

Space Complexity: O(N) as we are using a map data structure.

⚡⚡ 4. Sliding Window (Two Pointers) — Only for Positive Numbers

💡 Idea

Since all numbers are positive,
we can use a sliding window that expands and shrinks intelligently.

- Add elements to window (increase `right`)
- If sum > k → shrink from left
- If sum == k → check length

```
#include <bits/stdc++.h>
using namespace std;

int getLongestSubarray(vector<int>& a, long long k) {
    int n = a.size(); // size of the array.

    int left = 0, right = 0; // 2 pointers
    long long sum = a[0];
    int maxLen = 0;
    while (right < n) {
        // if sum > k, reduce the subarray from left
        // until sum becomes less or equal to k:
        while (left <= right && sum > k) {
            sum -= a[left];
            left++;
        }

        // if sum = k, update the maxLen i.e. answer:
        if (sum == k) {
            maxLen = max(maxLen, right - left + 1);
        }

        // Move forward thw right pointer:
        right++;
        if (right < n) sum += a[right];
    }

    return maxLen;
}
```

```
int main()
{
    vector<int> a = {2, 3, 5, 1, 9};
    long long k = 10;
    int len = getLongestSubarray(a, k);
    cout << "The length of the longest subarray is: " << len << "\n";
    return 0;
}
```

Time Complexity: O(2*N), where N = size of the given array.
Reason: The outer while loop i.e. the right pointer can move up to index n-1(the last index). Now, the inner while loop i.e. the left pointer can move up to the right pointer at most. So, every time the inner loop does not run for n times rather it can run for n times in total. So, the time complexity will be O(2*N) instead of O(N2).

Space Complexity: O(1) as we are not using any extra space.

Space Complexity: O(1) as we are not using any extra space.

14. Longest Subarray with sum K | [Postives and Negatives]

Brute Force

```
#include <bits/stdc++.h>
using namespace std;

int getLongestSubarray(vector<int>& a, int k) {
    int n = a.size(); // size of the array.

    int len = 0;
    for (int i = 0; i < n; i++) { // starting index
        for (int j = i; j < n; j++) { // ending index
            // add all the elements of
```

```
            // subarray = a[i...j]:
            int s = 0;
            for (int K = i; K <= j; K++) {
                s += a[K];
            }

            if (s == k)
                len = max(len, j - i + 1);
        }
    }
    return len;
}

int main()
{
    vector<int> a = { -1, 1, 1};
    int k = 1;
    int len = getLongestSubarray(a, k);
    cout << "The length of the longest subarray is: " << len << "\n";
    return 0;
}
```

Time Complexity: O(N3) approx., where N = size of the array.
Reason: We are using three nested loops, each running approximately N times.

Space Complexity: O(1) as we are not using any extra space.

<u>Brute force se thoda behtar</u>

```
#include <bits/stdc++.h>
using namespace std;

int getLongestSubarray(vector<int>& a, int k) {
    int n = a.size(); // size of the array.

    int len = 0;
    for (int i = 0; i < n; i++) { // starting index
        int s = 0;
        for (int j = i; j < n; j++) { // ending index
            // add the current element to
```

```
            // the subarray a[i...j-1]:
            s += a[j];

            if (s == k)
                len = max(len, j - i + 1);
        }
    }
    return len;
}

int main()
{
    vector<int> a = { -1, 1, 1};
    int k = 1;
    int len = getLongestSubarray(a, k);
    cout << "The length of the longest subarray is: " << len << "\n";
    return 0;
}
```

Time Complexity: O(N2) approx., where N = size of the array.
Reason: We are using two nested loops, each running approximately N times.

Space Complexity: O(1) as we are not using any extra space.

✅ Optimal code (use `unordered_map` for O(N) avg)

🔑 Core idea (Prefix sum + map)

Let `pref[i] = a[0] + a[1] + ... + a[i]`.

Agar kisi index `i` par `pref[i] = S` hai, aur pehle kabhi `pref[j] = S - K` mila tha (j < i),
to subarray `(j+1 ... i)` ka sum K hoga (kyunki `pref[i] - pref[j] = K`).

Isi liye:

- Har prefix sum ka pehla index map me store karo (max length chahiye, isliye first occurrence hi best hota hai).
- Har step par `rem = pref - K` dekho: agar rem pehle dikha, to length = `i - firstIndex[rem]`.

```cpp
#include <bits/stdc++.h>
using namespace std;

int getLongestSubarray(vector<int>& a, int k) {
    int n = a.size(); // size of the array.

    map<int, int> preSumMap;
    int sum = 0;
    int maxLen = 0;
    for (int i = 0; i < n; i++) {
        //calculate the prefix sum till index i:
        sum += a[i];

        // if the sum = k, update the maxLen:
        if (sum == k) {
            maxLen = max(maxLen, i + 1);
        }

        // calculate the sum of remaining part i.e. x-k:
        int rem = sum - k;

        //Calculate the length and update maxLen:
        if (preSumMap.find(rem) != preSumMap.end()) {
            int len = i - preSumMap[rem];
            maxLen = max(maxLen, len);
        }

        //Finally, update the map checking the conditions:
        if (preSumMap.find(sum) == preSumMap.end()) {
            preSumMap[sum] = i;
        }
    }

    return maxLen;
}

int main()
{
    vector<int> a = { -1, 1, 1};
    int k = 1;
    int len = getLongestSubarray(a, k);
    cout << "The length of the longest subarray is: " << len << "\n";
    return 0;
}
```

Time Complexity: O(N) or O(N*logN) depending on which map data structure we are using, where N = size of the array.
Reason: For example, if we are using an unordered_map data structure in C++ the time complexity will be O(N)(though in the worst case, unordered_map takes O(N) to find an element and the time complexity becomes O(N2)) but if we are using a map data structure, the time complexity will be O(N*logN). The least complexity will be O(N) as we are using a loop to traverse the array.

Space Complexity: O(N) as we are using a map data structure.

15. Two Sum : Check if a pair with given sum exists in Array

Input:
arr = [2, 6, 5, 8, 11]
target = 14

Output: YES
(6 + 8 = 14)

🐢 Approach 1: Brute Force (Nested Loops)

💡 Idea

Check every pair `(i, j)` and see if their sum equals `target`.

```
#include <bits/stdc++.h>
using namespace std;

string twoSum(int n, vector<int> &arr, int target) {
```

```
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[i] + arr[j] == target) return "YES";
        }
    }
    return "NO";
}

int main()
{
    int n = 5;
    vector<int> arr = {2, 6, 5, 8, 11};
    int target = 14;
    string ans = twoSum(n, arr, target);
    cout << "This is the answer for variant 1: " << ans << endl;
    return 0;
}
```

Time Complexity: O(N2), where N = size of the array.
Reason: There are two loops(i.e. nested) each running for approximately N times.

Space Complexity: O(1) as we are not using any extra space.

—----------

Variety 2:

```
#include <bits/stdc++.h>
using namespace std;

vector<int> twoSum(int n, vector<int> &arr, int target) {
    vector<int> ans;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[i] + arr[j] == target) {
                ans.push_back(i);
                ans.push_back(j);
                return ans;
            }
        }
    }
```

```
    return { -1, -1};
}

int main()
{
    int n = 5;
    vector<int> arr = {2, 6, 5, 8, 11};
    int target = 14;
    vector<int> ans = twoSum(n, arr, target);
    cout << "This is the answer for variant 2: [" << ans[0] << ", "
         << ans[1] << "]" << endl;
    return 0;
}
```

Time Complexity: O(N2), where N = size of the array.
Reason: There are two loops(i.e. nested) each running for approximately N times.

Space Complexity: O(1) as we are not using any extra space.

⚡ Approach 2: Hash Map (Optimal)

💡 Idea

As you traverse the array, store each element in a map.
For each number `num`, check if `target - num` exists in the map.
If yes → found a pair.

```
#include <bits/stdc++.h>
using namespace std;

string twoSum(int n, vector<int> &arr, int target) {
    unordered_map<int, int> mpp;
    for (int i = 0; i < n; i++) {
        int num = arr[i];
        int moreNeeded = target - num;
        if (mpp.find(moreNeeded) != mpp.end()) {
            return "YES";
        }
```

```
        mpp[num] = i;
    }
    return "NO";
}

int main()
{
    int n = 5;
    vector<int> arr = {2, 6, 5, 8, 11};
    int target = 14;
    string ans = twoSum(n, arr, target);
    cout << "This is the answer for variant 1: " << ans << endl;
    return 0;
}
```

Time Complexity: O(N), where N = size of the array.
Reason: The loop runs N times in the worst case and searching in a hashmap takes O(1) generally. So the time complexity is O(N).

Note: In the worst case(which rarely happens), the unordered_map takes O(N) to find an element. In that case, the time complexity will be O(N2). If we use map instead of unordered_map, the time complexity will be O(N* logN) as the map data structure takes logN time to find an element.

Space Complexity: O(N) as we use the map data structure.

Note: We have optimized this problem enough. But if in the interview, we are not allowed to use the map data structure, then we should move on to the following approach i.e. two pointer approach. This approach will have the same time complexity as the better approach.

Variety 2:

```
#include <bits/stdc++.h>
using namespace std;

vector<int> twoSum(int n, vector<int> &arr, int target) {
    unordered_map<int, int> mpp;
    for (int i = 0; i < n; i++) {
        int num = arr[i];
        int moreNeeded = target - num;
        if (mpp.find(moreNeeded) != mpp.end()) {
            return {mpp[moreNeeded], i};
        }
```

```
        mpp[num] = i;
    }
    return { -1, -1};
}

int main()
{
    int n = 5;
    vector<int> arr = {2, 6, 5, 8, 11};
    int target = 14;
    vector<int> ans = twoSum(n, arr, target);
    cout << "This is the answer for variant 2: [" << ans[0] << ", "
         << ans[1] << "]" << endl;
    return 0;
}
```

Time Complexity: O(N), where N = size of the array.
Reason: The loop runs N times in the worst case and searching in a hashmap takes O(1) generally. So the time complexity is O(N).

Note: In the worst case(which rarely happens), the unordered_map takes O(N) to find an element. In that case, the time complexity will be O(N2). If we use map instead of unordered_map, the time complexity will be O(N* logN) as the map data structure takes logN time to find an element.

Space Complexity: O(N) as we use the map data structure.

🧭 Approach 3: Two Pointer (Sorted Array)

💡 Idea

If the array is sorted:

- Start one pointer at beginning (`left`)
- Another at end (`right`)
- If sum < target → move left++
- If sum > target → move right--
- If sum == target → return YES

```
#include <bits/stdc++.h>
using namespace std;

string twoSum(int n, vector<int> &arr, int target) {
    sort(arr.begin(), arr.end());
    int left = 0, right = n - 1;
    while (left < right) {
        int sum = arr[left] + arr[right];
        if (sum == target) {
            return "YES";
        }
        else if (sum < target) left++;
        else right--;
    }
    return "NO";
}

int main()
{
    int n = 5;
    vector<int> arr = {2, 6, 5, 8, 11};
    int target = 14;
    string ans = twoSum(n, arr, target);
    cout << "This is the answer for variant 1: " << ans << endl;
    return 0;
}
```

Note: For variant 2, we can store the elements of the array along with its index in a new array. Then the rest of the code will be similar. And while returning, we need to return the stored indices instead of returning “YES”. But for this variant, the recommended approach is approach 2 i.e. hashing approach.

Time Complexity: O(N) + O(N*logN), where N = size of the array.
Reason: The loop will run at most N times. And sorting the array will take N*logN time complexity.

Space Complex: O(1)

16. Sort an array of 0s, 1s and 2s

Sorting (even if it is not the expected solution here but it can be considered as one of the approaches). Since the array contains only 3 integers, 0, 1, and 2. Simply sorting the array would arrange the elements in increasing order.

Complexity Analysis

Time Complexity: O(N*logN)

Space Complexity: O(1)

🪜 Approach 2: Counting Method (a.k.a. Counting Sort for 3 Elements)

💡 Idea:

Count how many 0s, 1s, and 2s exist,
then rewrite the array based on these counts.

```
#include <bits/stdc++.h>
using namespace std;

void sortArray(vector<int>& arr, int n) {

    int cnt0 = 0, cnt1 = 0, cnt2 = 0;

    for (int i = 0; i < n; i++) {
        if (arr[i] == 0) cnt0++;
        else if (arr[i] == 1) cnt1++;
        else cnt2++;
    }

    //Replace the places in the original array:
    for (int i = 0; i < cnt0; i++) arr[i] = 0; // replacing 0's

    for (int i = cnt0; i < cnt0 + cnt1; i++) arr[i] = 1; // replacing 1's

    for (int i = cnt0 + cnt1; i < n; i++) arr[i] = 2; // replacing 2's
```

```
}

int main()
{
    int n = 6;
    vector<int> arr = {0, 2, 1, 2, 0, 1};
    sortArray(arr, n);
    cout << "After sorting:" << endl;
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}
```

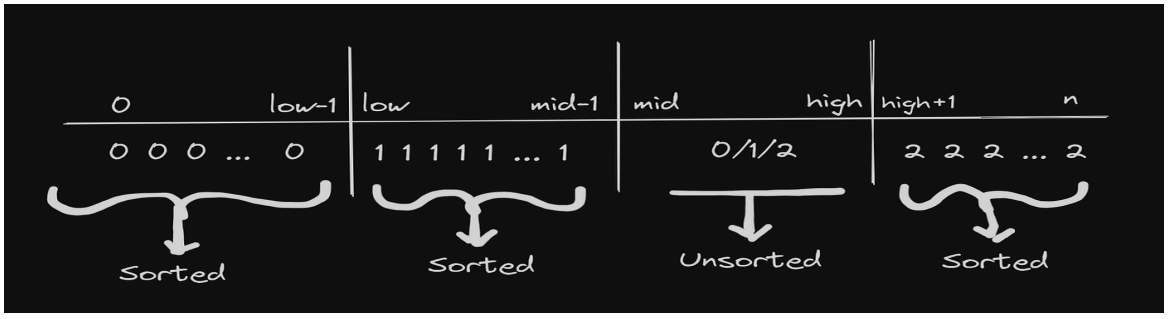
Output:
After sorting:
0 0 1 1 2 2

Time Complexity: O(N) + O(N), where N = size of the array. First O(N) for counting the number of 0's, 1's, 2's, and second O(N) for placing them correctly in the original array.

Space Complexity: O(1) as we are not using any extra space.

🚀 Approach 3: Dutch National Flag Algorithm (Most Optimal)

This is the **most efficient and elegant solution** — single traversal, in-place, O(1) space.



```
#include <bits/stdc++.h>
using namespace std;

void sortArray(vector<int>& arr, int n) {

    int low = 0, mid = 0, high = n - 1; // 3 pointers

    while (mid <= high) {
        if (arr[mid] == 0) {
            swap(arr[low], arr[mid]);
            low++;
            Mid++; //0 k sath 1 ko bhi shi jagah lena hai
        }
        else if (arr[mid] == 1) {
            mid++;
        }
        else {
            swap(arr[mid], arr[high]);
            high--;
        }
    }
}
int main()
{
    int n = 6;
    vector<int> arr = {0, 2, 1, 2, 0, 1};
    sortArray(arr, n);
    cout << "After sorting:" << endl;
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}
```

Output:
After sorting:
0 0 1 1 2 2

Time Complexity: O(N), where N = size of the given array.
Reason: We are using a single loop that can run at most N times.

Space Complexity: O(1) as we are not using any extra space.

17. Find the Majority Element that occurs more than N/2 times

1) Brute force (nested loops)

```
#include <bits/stdc++.h>
using namespace std;

int majorityElement(vector<int> v) {

    //size of the given array:
    int n = v.size();

    for (int i = 0; i < n; i++) {
        //selected element is v[i]
        int cnt = 0;
        for (int j = 0; j < n; j++) {
            // counting the frequency of v[i]
            if (v[j] == v[i]) {
                cnt++;
            }
        }

        // check if frquency is greater than n/2:
        if (cnt > (n / 2))
            return v[i];
    }

    return -1;
}

int main()
{
    vector<int> arr = {2, 2, 1, 1, 1, 2, 2};
    int ans = majorityElement(arr);
    cout << "The majority element is: " << ans << endl;
    return 0;
}
```

Output: The majority element is: 2

Time Complexity: O(N2), where N = size of the given array. Reason: For every element of the array the inner loop runs for N times. And there are N elements in the array. So, the total time complexity is O(N2).
Space Complexity: O(1) as we use no extra space.

2) Hash/map counting

```
#include <bits/stdc++.h>
using namespace std;

int majorityElement(vector<int> v) {

    //size of the given array:
    int n = v.size();

    //declaring a map:
    map<int, int> mpp;

    //storing the elements with its occurnce:
    for (int i = 0; i < n; i++) {
        mpp[v[i]]++;
    }

    //searching for the majority element:
    for (auto it : mpp) {
        if (it.second > (n / 2)) {
            return it.first;
        }
    }

    return -1;
}

int main()
{
    vector<int> arr = {2, 2, 1, 1, 1, 2, 2};
    int ans = majorityElement(arr);
    cout << "The majority element is: " << ans << endl;
    return 0;
}
```

Output: The majority element is: 2

Time Complexity: O(N*logN) + O(N), where N = size of the given array.
Reason: We are using a map data structure. Insertion in the map takes logN time. And we are doing it for N elements. So, it results in the first term O(N*logN). The second O(N) is for checking which element occurs more than floor(N/2) times. If we use unordered_map instead, the first term will be O(N) for the best and average case and for the worst case, it will be O(N2).

Space Complexity: O(N) as we are using a map data structure.

3) Boyer–Moore Majority Vote (⭐ optimal)

```
#include <bits/stdc++.h>
using namespace std;

int majorityElement(vector<int> v) {

    //size of the given array:
    int n = v.size();
    int cnt = 0; // count
    int el; // Element

    //applying the algorithm:
    for (int i = 0; i < n; i++) {
        if (cnt == 0) {
            cnt = 1;
            el = v[i];
        }
        else if (el == v[i]) cnt++;
        else cnt--;
    }

    //checking if the stored element
    // is the majority element:
    int cnt1 = 0;
    for (int i = 0; i < n; i++) {
        if (v[i] == el) cnt1++;
    }
```

```
    if (cnt1 > (n / 2)) return el;
    return -1;
}

int main()
{
    vector<int> arr = {2, 2, 1, 1, 1, 2, 2};
    int ans = majorityElement(arr);
    cout << "The majority element is: " << ans << endl;
    return 0;
}
```

Output: The majority element is: 2

Time Complexity: O(N) + O(N), where N = size of the given array.
Reason: The first O(N) is to calculate the count and find the expected majority element. The second one is to check if the expected element is the majority one or not.

Note: If the question states that the array must contain a majority element, in that case, we do not need the second check. Then the time complexity will boil down to O(N).

Space Complexity: O(1) as we are not using any extra space.

18. Kadane's Algorithm : Maximum Subarray Sum in an Array

Input: [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Output: 6
Explanation: The subarray [4, -1, 2, 1] has the largest sum = 6

- **Brute Force (3 loops)**

  1. Generate every subarray `(i...j)`.
  2. Compute its sum using a third loop.
  3. Track the maximum sum.

```
#include <bits/stdc++.h>
using namespace std;

int maxSubarraySum(int arr[], int n) {
    int maxi = INT_MIN; // maximum sum

    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            // subarray = arr[i.....j]
            int sum = 0;

            //add all the elements of subarray:
            for (int k = i; k <= j; k++) {
                sum += arr[k];
            }

            maxi = max(maxi, sum);
        }
    }

    return maxi;
}

int main()
{
    int arr[] = { -2, 1, -3, 4, -1, 2, 1, -5, 4};
    int n = sizeof(arr) / sizeof(arr[0]);
    int maxSum = maxSubarraySum(arr, n);
    cout << "The maximum subarray sum is: " << maxSum << endl;
    return 0;
}
```

Output: The maximum subarray sum is: 6

Time Complexity: O(N3), where N = size of the array.
Reason: We are using three nested loops, each running approximately N times.

Space Complexity: O(1) as we are not using any extra space.

🔹 Improved (2 loops)

Reuse previously computed subarray sums to avoid one inner loop.

```
#include <bits/stdc++.h>
using namespace std;

int maxSubarraySum(int arr[], int n) {
    int maxi = INT_MIN; // maximum sum

    for (int i = 0; i < n; i++) {
        int sum = 0;
        for (int j = i; j < n; j++) {
            // current subarray = arr[i.....j]

            //add the current element arr[j]
            // to the sum i.e. sum of arr[i...j-1]
            sum += arr[j];

            maxi = max(maxi, sum); // getting the maximum
        }
    }

    return maxi;
}

int main()
{
    int arr[] = { -2, 1, -3, 4, -1, 2, 1, -5, 4};
    int n = sizeof(arr) / sizeof(arr[0]);
    int maxSum = maxSubarraySum(arr, n);
    cout << "The maximum subarray sum is: " << maxSum << endl;
    return 0;
}
```

Output: The maximum subarray sum is: 6

Time Complexity: O(N2), where N = size of the array.
Reason: We are using two nested loops, each running approximately N times.

Space Complexity: O(1) as we are not using any extra space.

- **Kadane's Algorithm (O(N))**

At any element `arr[i]`, we have two choices:

1. Continue the previous subarray → `sum + arr[i]`
2. Start a new subarray → `arr[i]`

Whichever gives the **larger sum**, we choose that.

Also, if at any point the running sum becomes **negative**,
we reset it to 0 — because a negative prefix can never help future sums.

```
#include <bits/stdc++.h>
using namespace std;

long long maxSubarraySum(int arr[], int n) {
    long long maxi = LONG_MIN; // maximum sum
    long long sum = 0;

    for (int i = 0; i < n; i++) {

        sum += arr[i];

        if (sum > maxi) {
            maxi = sum;
        }

        // If sum < 0: discard the sum calculated
        if (sum < 0) {
            sum = 0;
        }
    }

    // To consider the sum of the empty subarray
    // uncomment the following check:

    //if (maxi < 0) maxi = 0;

    return maxi;
}

int main()
{
```

```
    int arr[] = { -2, 1, -3, 4, -1, 2, 1, -5, 4};
    int n = sizeof(arr) / sizeof(arr[0]);
    long long maxSum = maxSubarraySum(arr, n);
    cout << "The maximum subarray sum is: " << maxSum << endl;
    return 0;
}
```

Output: The maximum subarray sum is: 6

Time Complexity: O(N), where N = size of the array.
Reason: We are using a single loop running N times.

Space Complexity: O(1) as we are not using any extra space.

🎯 To Print the Subarray Too

```
#include <bits/stdc++.h>
using namespace std;

long long maxSubarraySum(int arr[], int n) {
    long long maxi = LONG_MIN; // maximum sum
    long long sum = 0;

    int start = 0;
    int ansStart = -1, ansEnd = -1;
    for (int i = 0; i < n; i++) {

        if (sum == 0) start = i; // starting index

        sum += arr[i];

        if (sum > maxi) {
            maxi = sum;

            ansStart = start;
            ansEnd = i;
        }

        // If sum < 0: discard the sum calculated
        if (sum < 0) {
            sum = 0;
        }
```

```
    }

    //printing the subarray:
    cout << "The subarray is: [";
    for (int i = ansStart; i <= ansEnd; i++) {
        cout << arr[i] << " ";
    }
    cout << "]n";

    // To consider the sum of the empty subarray
    // uncomment the following check:

    //if (maxi < 0) maxi = 0;

    return maxi;
}

int main()
{
    int arr[] = { -2, 1, -3, 4, -1, 2, 1, -5, 4};
    int n = sizeof(arr) / sizeof(arr[0]);
    long long maxSum = maxSubarraySum(arr, n);
    cout << "The maximum subarray sum is: " << maxSum << endl;
    return 0;
}
```

Output:
The subarray is: [4 -1 2 1]
The maximum subarray sum is: 6

Time Complexity: O(N), where N = size of the array.
Reason: We are using a single loop running N times.

Space Complexity: O(1) as we are not using any extra space.

20. Stock Buy And Sell

Given an array `arr[]` where each element represents the stock price on day `i`,
find the **maximum profit** you can achieve by buying and selling **once**.

Input: [7, 1, 5, 3, 6, 4]
Output: 5
Explanation:
Buy on day 2 (price = 1), sell on day 5 (price = 6)
Profit = 6 - 1 = 5

Approach 1: Brute Force (O(N²))

- Try **all pairs (i, j)** such that `j > i`.
- Compute profit = `arr[j] - arr[i]`
- Keep track of the maximum profit.

```
#include<bits/stdc++.h>
using namespace std;

int maxProfit(vector<int> &arr) {
    int maxPro = 0;
    int n = arr.size();

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[j] > arr[i]) {
            maxPro = max(arr[j] - arr[i], maxPro);
            }
        }
        }

    return maxPro;
}

int main() {
    vector<int> arr = {7,1,5,3,6,4};
    int maxPro = maxProfit(arr);
    cout << "Max profit is: " << maxPro << endl;
}
```

Output:

Max profit is: 5

Time complexity: O(n^2)

Space Complexity: O(1)

Approach 2: Optimized (O(N)) — Kadane's style

Now think smartly 👇
Instead of checking every pair, we can do this in **one pass**.

💡 **Intuition**

- As we traverse, keep track of the **lowest price so far** (the best day to buy).
- For each price, calculate the **potential profit** = `price - minPriceSoFar`.
- Keep track of the **maximum profit**.

```
#include<bits/stdc++.h>
using namespace std;

int maxProfit(vector<int> &arr) {
    int maxPro = 0;
    int n = arr.size();
    int minPrice = INT_MAX;

    for (int i = 0; i < arr.size(); i++) {
        minPrice = min(minPrice, arr[i]);
        maxPro = max(maxPro, arr[i] - minPrice);
    }

    return maxPro;
}

int main() {
    vector<int> arr = {7,1,5,3,6,4};
    int maxPro = maxProfit(arr);
```

```
    cout << "Max profit is: " << maxPro << endl;
}
```

Output: Max profit is: 5

Time complexity: O(n)

Space Complexity: O(1)

21. Rearrange Array Elements by Sign

- Rearrange array so **positives and negatives alternate**.
- If counts are **equal** → strictly alternate: `+ - + - ...`
- If counts are **unequal** → alternate as much as possible, then **append leftovers** of the majority sign at the end.

  🔎 Decide how to treat **0**. Our code treats `0` as **negative** (`A[i] > 0 ? pos : neg`).
  Many problems either treat `0` as positive or allow either. Be consistent with the statement.

Case A: Counts are equal (strict alternation)

A1) Two-bucket, write back (simple & readable)

```
#include<bits/stdc++.h>
using namespace std;

  vector<int> RearrangebySign(vector<int>A, int n){

  // Define 2 vectors, one for storing positive
  // and other for negative elements of the array.
  vector<int> pos;
  vector<int> neg;
```

```
  // Segregate the array into positives and negatives.
  for(int i=0;i<n;i++){

     if(A[i]>0) pos.push_back(A[i]);
     else neg.push_back(A[i]);
  }

  // Positives on even indices, negatives on odd.
  for(int i=0;i<n/2;i++){

     A[2*i] = pos[i];
     A[2*i+1] = neg[i];
  }


  return A;

}

int main() {

  // Array Initialisation.
  int n = 4;
  vector<int> A {1,2,-4,-5};

  vector<int> ans = RearrangebySign(A,n);

  for (int i = 0; i < ans.size(); i++) {
    cout << ans[i] << " ";
  }

  return 0;
}
```

Output :

1 -4 2 -5

Time Complexity: O(N+N/2) { O(N) for traversing the array once for segregating positives and negatives and another O(N/2) for adding those elements alternatively to the array, where N = size of the array A}.

Space Complexity: O(N/2 + N/2) = O(N) { N/2 space required for each of the positive and negative element arrays, where N = size of the array A}.

A2) Single pass fill even/odd (no extra buckets for counts)

```
#include<bits/stdc++.h>
using namespace std;

  vector<int> RearrangebySign(vector<int>A){

  int n = A.size();

  // Define array for storing the ans separately.
  vector<int> ans(n,0);

  // positive elements start from 0 and negative from 1.
  int posIndex = 0, negIndex = 1;
  for(int i = 0;i<n;i++){

      // Fill negative elements in odd indices and inc by 2.
      if(A[i]<0){
          ans[negIndex] = A[i];
          negIndex+=2;
      }

      // Fill positive elements in even indices and inc by 2.
      else{
          ans[posIndex] = A[i];
          posIndex+=2;
      }
  }

  return ans;

}

int main() {

  // Array Initialisation.

  vector<int> A = {1,2,-4,-5};

  vector<int> ans = RearrangebySign(A);
```

```
  for (int i = 0; i < ans.size(); i++) {
    cout << ans[i] << " ";
  }

  return 0;
}
```

Output :

1 -4 2 -5

Time Complexity: O(N) { O(N) for traversing the array once and substituting positives and negatives simultaneously using pointers, where N = size of the array A}.

Space Complexity: O(N) { Extra Space used to store the rearranged elements separately in an array, where N = size of array A}.

Case B: Counts may be unequal

Strategy:

1. Split into `pos` and `neg`.
2. Fill alternating while both remain.
3. Append the leftover of the majority sign.

```
#include<bits/stdc++.h>
using namespace std;

  vector<int> RearrangebySign(vector<int>A, int n){

  // Define 2 vectors, one for storing positive
  // and other for negative elements of the array.
  vector<int> pos;
  vector<int> neg;

  // Segregate the array into positives and negatives.
  for(int i=0;i<n;i++){

     if(A[i]>0) pos.push_back(A[i]);
     else neg.push_back(A[i]);
```

```
  }

  // If positives are lesser than the negatives.
  if(pos.size() < neg.size()){

    // First, fill array alternatively till the point
    // where positives and negatives ar equal in number.
    for(int i=0;i<pos.size();i++){

      A[2*i] = pos[i];
      A[2*i+1] = neg[i];
    }

    // Fill the remaining negatives at the end of the array.
    int index = pos.size()*2;
    for(int i = pos.size();i<neg.size();i++){

        A[index] = neg[i];
        index++;
    }
  }

  // If negatives are lesser than the positives.
  else{

      // First, fill array alternatively till the point
      // where positives and negatives ar equal in number.
      for(int i=0;i<neg.size();i++){

      A[2*i] = pos[i];
      A[2*i+1] = neg[i];
  }

    // Fill the remaining positives at the end of the array.
    int index = neg.size()*2;
    for(int i = neg.size();i<pos.size();i++){

        A[index] = pos[i];
        index++;
    }
  }
  return A;

}
```

```
int main() {

  // Array Initialisation.
  int n = 6;
  vector<int> A {1,2,-4,-5,3,4};

  vector<int> ans = RearrangebySign(A,n);

  for (int i = 0; i < ans.size(); i++) {
    cout << ans[i] << " ";
  }

  return 0;
}
```
Output :

1 -4 2 -5 3 4

Time Complexity: O(2*N) { The worst case complexity is O(2*N) which is a combination of O(N) of traversing the array to segregate into neg and pos array and O(N) for adding the elements alternatively to the main array}.

Explanation: The second O(N) is a combination of O(min(pos, neg)) + O(leftover elements). There can be two cases: when only positive or only negative elements are present, O(min(pos, neg)) + O(leftover) = O(0) + O(N), and when equal no. of positive and negative elements are present, O(min(pos, neg)) + O(leftover) = O(N/2) + O(0). So, from these two cases, we can say the worst-case time complexity is O(N) for the second part, and by adding the first part we get the total complexity of O(2*N).

Space Complexity: O(N/2 + N/2) = O(N) { N/2 space required for each of the positive and negative element arrays, where N = size of the array A}.

22. next_permutation : find next lexicographically greater permutation

Given a sequence, find the **next** arrangement in dictionary (lexicographic) order.
If you're already at the largest arrangement (strictly decreasing), the next is the **smallest** (sorted ascending).

Brute-Force Approach

The brute force approach to find the next permutation is to find all possible permutations of the array and then look for next permutation.

- Find all possible permutations of elements present and store them.
- Sort the permutations and search input from all possible permutations. Print the next permutation present right after it.If the current permutation is the last, return the first permutation in the list.

Recursion use karenge to generate all the permutation:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

void generatePermutations(vector<vector<int>> &res,
                          vector<int> &arr, int idx) {

    // Base case: if idx reaches the end of array
    if (idx == arr.size() - 1) {
        res.push_back(arr);
        return;
    }

    // Generate all permutations by swapping
    for (int i = idx; i < arr.size(); i++) {
        swap(arr[idx], arr[i]);

        // Recur for the next index
        generatePermutations(res, arr, idx + 1);

        // Backtrack to restore original array
        swap(arr[idx], arr[i]);
    }
```

```
}

// Function to find the next permutation
void nextPermutation(vector<int>& arr) {

    vector<vector<int>> res;

    // Generate all permutations
    generatePermutations(res, arr, 0);

    // Sort all permutations lexicographically
    sort(res.begin(), res.end());

    // Find the current permutation index
    for (int i = 0; i < res.size(); i++) {

        // If current permutation matches input
        if (res[i] == arr) {

            // If it's not the last permutation
            if (i < res.size() - 1) {
                arr = res[i + 1];
            }

            // If it's the last permutation
            else {
                arr = res[0];
            }

            break;
        }
    }
}

int main() {

    vector<int> arr = {2, 4, 1, 7, 5, 0};

    nextPermutation(arr);

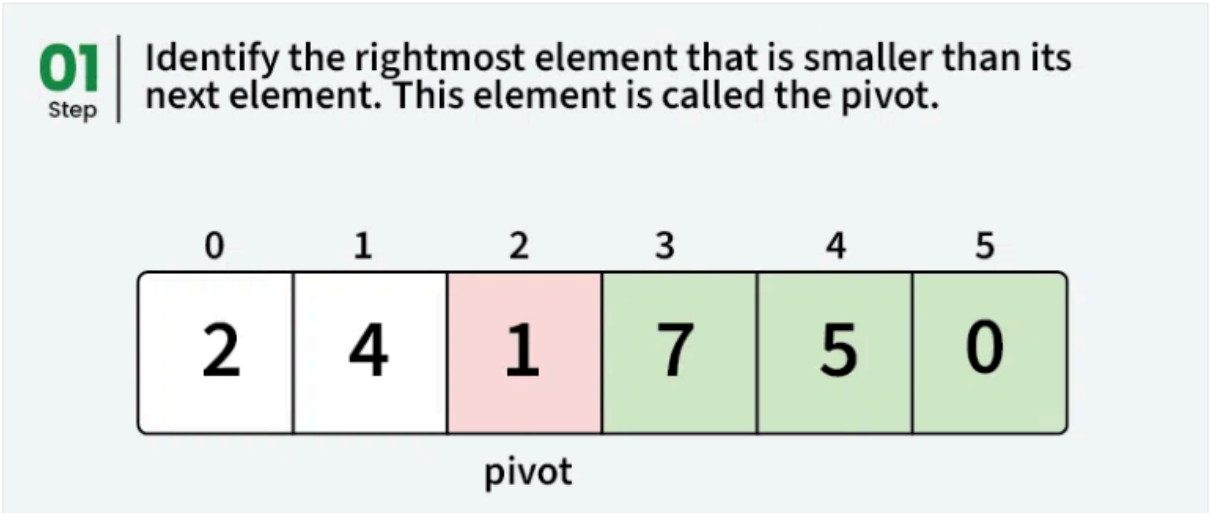
    for (int i = 0; i < arr.size(); i++) {
        cout << arr[i] << " ";
    }
```

```
    return 0;
}
```

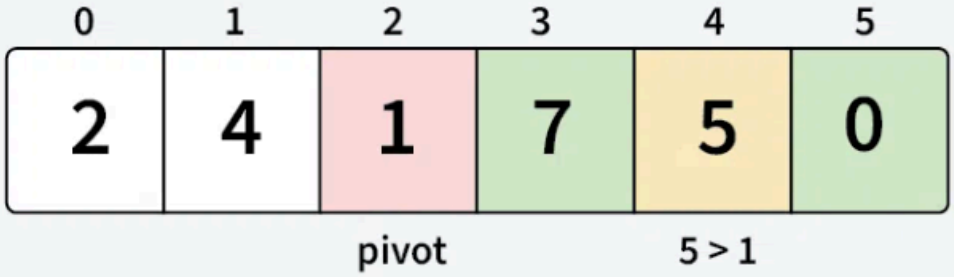
Optimal Approach

Think of the array as: **prefix | pivot | suffix**

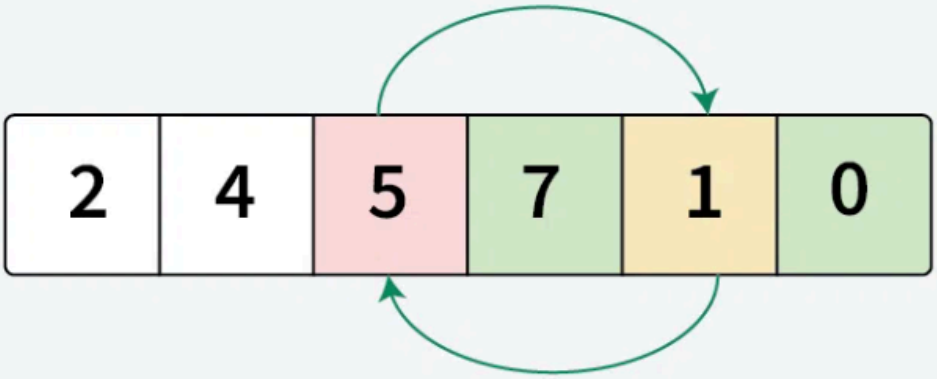
1. **Scan from right to left** to find the first index `i` (pivot) such that `nums[i] < nums[i+1]`.
    - If no such `i` exists, the array is in descending order → reverse entire array and return.
2. **Find the smallest number > nums[i]** in the suffix (which is decreasing), so just scan from right to left to find the first `j` with `nums[j] > nums[i]`.
3. **Swap** `nums[i]` and `nums[j]`.
4. **Reverse** the suffix (from `i+1` to end).
   Why reverse? Because the suffix was decreasing; reversing makes it the **smallest** ascending tail after the new pivot → gives the immediate next permutation.

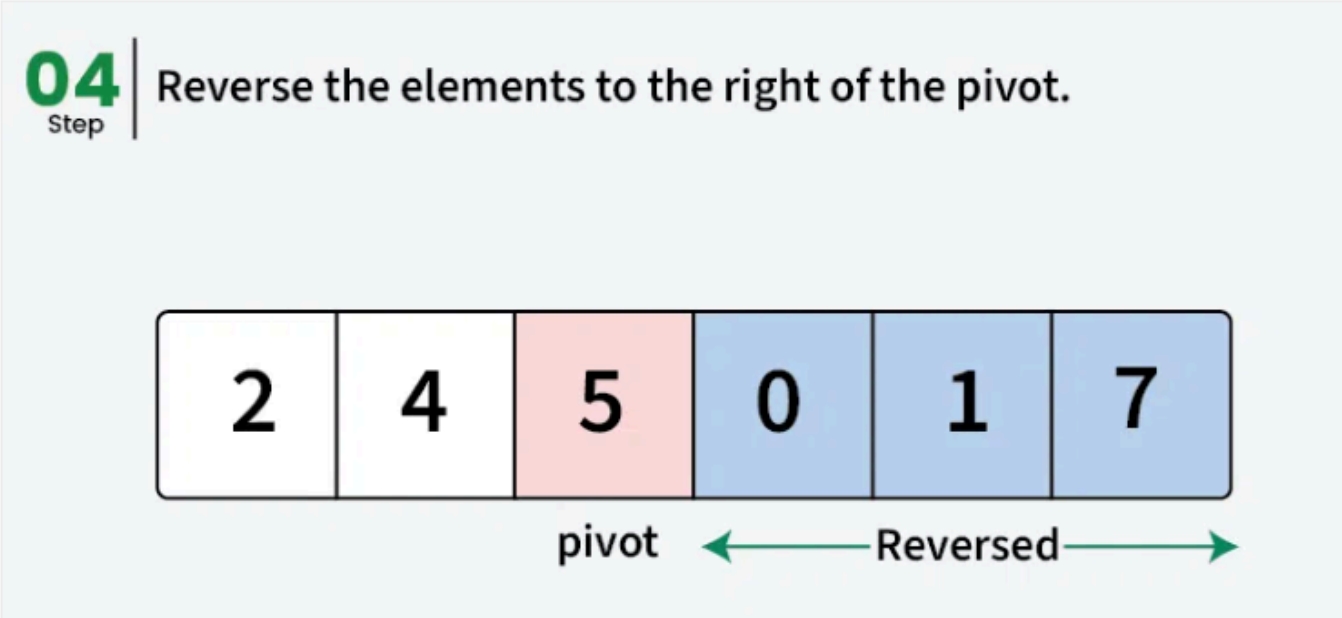


02 Step | Find the rightmost element that is greater than the pivot.



03 Step | Swap Pivot and Rightmost Greater Element






```
#include <bits/stdc++.h>
using namespace std;

// Solution class
class Solution {
public:
    // Function to find next permutation
    void nextPermutation(vector<int>& nums) {
        // Set index to -1
        int index = -1;

        // Find the first decreasing element from end
        for (int i = nums.size() - 2; i >= 0; i--) {
            // If a smaller element found
            if (nums[i] < nums[i + 1]) {
                // Store index
                index = i;
                break;
            }
        }

        // If no such index found
        if (index == -1) {
            // Reverse the entire array
            reverse(nums.begin(), nums.end());
```

```
            return;
        }

        // Find element just greater than nums[index]
        for (int i = nums.size() - 1; i > index; i--) {
            // Swap the two
            if (nums[i] > nums[index]) {
                swap(nums[i], nums[index]);
                break;
            }
        }

        // Reverse the part after index
        reverse(nums.begin() + index + 1, nums.end());
    }
};

// Main function
int main() {
    // Input array
    vector<int> nums = {1, 2, 3};

    // Create object
    Solution sol;

    // Call the function
    sol.nextPermutation(nums);

    // Print result
    for (int num : nums) {
        cout << num << " ";
    }
    cout << endl;

    return 0;
}
```

Complexity Analysis
Time Complexity: O(N), we find the breaking point and reverse the subarray in linear time.
Space Complexity: O(1), constant additional space is used.

🔧 Bonus: STL one-liner

C++ already gives you this algorithm!

```
vector<int> nums = {1,2,3};
if (!next_permutation(nums.begin(), nums.end())) {
    // nums has become the smallest permutation
}
```

It returns `false` if it wrapped around (i.e., you were at the last permutation).

23. Leaders in an Array

Given an array, an element is called a **Leader** if **it is strictly greater than all elements to its right**.

✅ The **last element** of every array is **always** a leader (since there's nothing to its right).

arr = [10, 22, 12, 3, 0, 6]
Leaders → 22, 12, 6

🧠 Approach 1: Brute Force

💡 Idea:

For each element, check **every element to its right** —
if you find any greater element, it's **not a leader**.

```
#include<bits/stdc++.h>
using namespace std;

vector<int> printLeadersBruteForce(int arr[], int n) {
```

```
  vector<int> ans;

  for (int i = 0; i < n; i++) {
    bool leader = true;

    //Checking whether arr[i] is greater than all
    //the elements in its right side
    for (int j = i + 1; j < n; j++)
      if (arr[j] > arr[i]) {

        // If any element found is greater than current leader
        // curr element is not the leader.
        leader = false;
        break;
      }

    // Push all the leaders in ans array.
    if (leader)
    ans.push_back(arr[i]);

  }

  return ans;
}

int main() {

  // Array Initialization.
  int n = 6;
  int arr[n] = {10, 22, 12, 3, 0, 6};

  vector<int> ans = printLeadersBruteForce(arr,n);

  for(int i = 0;i<ans.size();i++){

      cout<<ans[i]<<" ";
  }

  cout<<endl;
  return 0;
}
```

Output:

22 12 6

Time Complexity: O(N^2) { Since there are nested loops being used, at the worst case n^2 time would be consumed }.

Space Complexity: O(N) { There is no extra space being used in this approach. But, a O(N) of space for ans array will be used in the worst case }.

⚡ Approach 2: **Optimal – Traverse from Right**

Since we only need elements greater than all to their **right**,
we can **traverse the array from right to left** while keeping track of the **current maximum**.

```
#include<bits/stdc++.h>
using namespace std;

vector<int> printLeaders(int arr[], int n) {

  vector<int> ans;

 // Last element of an array is always a leader,
 // push into ans array.
 int max = arr[n - 1];
 ans.push_back(arr[n-1]);

  // Start checking from the end whether a number is greater
  // than max no. from right, hence leader.
  for (int i = n - 2; i >= 0; i--)
    if (arr[i] > max) {
      ans.push_back(arr[i]);
      max = arr[i];
    }


  return ans;
}
```

```
int main() {

  // Array Initialization.
  int n = 6;
  int arr[n] = {10, 22, 12, 3, 0, 6};

  vector<int> ans = printLeaders(arr,n);


  for(int i = ans.size()-1;i>=0;i--){

      cout<<ans[i]<<" ";
  }

  cout<<endl;
  return 0;
}
```

Output:

22 12 6

Time Complexity: O(N) { Since the array is traversed single time back to front, it will consume O(N) of time where N = size of the array }.

Space Complexity: O(N) { There is no extra space being used in this approach. But, a O(N) of space for ans array will be used in the worst case }.

24. Longest Consecutive Sequence in an Array

Idea: Har element `x` ke liye check karo `x+1, x+2, ...` array me exist karte hain ya nahi (linearSearch).
Cons: Repeated linear search → **slow**.

Approach 1: Brute force (O(N²))

```
#include <bits/stdc++.h>
using namespace std;
```

```
bool linearSearch(vector<int>&a, int num) {
    int n = a.size(); //size of array
    for (int i = 0; i < n; i++) {
        if (a[i] == num)
            return true;
    }
    return false;
}
int longestSuccessiveElements(vector<int>&a) {
    int n = a.size(); //size of array
    int longest = 1;
    //pick a element and search for its
    //consecutive numbers:
    for (int i = 0; i < n; i++) {
        int x = a[i];
        int cnt = 1;
        //search for consecutive numbers
        //using linear search:
        while (linearSearch(a, x + 1) == true) {
            x += 1;
            cnt += 1;
        }

        longest = max(longest, cnt);
    }
    return longest;
}

int main()
{
    vector<int> a = {100, 200, 1, 2, 3, 4};
    int ans = longestSuccessiveElements(a);
    cout << "The longest consecutive sequence is " << ans << "\n";
    return 0;
}
```

Output: The longest consecutive sequence is 4.

Time Complexity: O(N2), N = size of the given array.
Reason: We are using nested loops each running for approximately N times.

Space Complexity: O(1), as we are not using any extra space to solve this problem.

Approach 2: Sort then scan (O(N log N))

Idea:

1. Array sort karo.
2. Ek pass me consecutive run length count karo.
3. **Duplicate elements** ko skip karna zaroori hai, warna count toot jayega.

```
#include <bits/stdc++.h>
using namespace std;

int longestConsecutive_sort(vector<int>& a) {
    int n = (int)a.size();
    if (n == 0) return 0;

    sort(a.begin(), a.end());
    int longest = 1, curr = 1;

    for (int i = 1; i < n; i++) {
        if (a[i] == a[i-1]) continue;          // duplicate → ignore
        if (a[i] == a[i-1] + 1) curr++;        // consecutive
        else { longest = max(longest, curr);   // break
               curr = 1; }
    }
    longest = max(longest, curr);
    return longest;
}
```

Time Complexity: O(NlogN) + O(N), N = size of the given array.
Reason: O(NlogN) for sorting the array. To find the longest sequence, we are using a loop that results in O(N).

Space Complexity: O(1), as we are not using any extra space to solve this problem.

Approach 3: Hash set (O(N) average)

Best for interviews.

Idea:

- `unordered_set` me sab elements daal do.
- Sirf **starting points** se sequences grow karo: koi number `x` tab start hoga jab `x-1` **set me na ho**.
- Fir `x+1, x+2, ...` count karte jao.

```
#include <bits/stdc++.h>
using namespace std;

int longestSuccessiveElements(vector<int>&a) {
    int n = a.size();
    if (n == 0) return 0;

    int longest = 1;
    unordered_set<int> st;
    //put all the array elements into set:
    for (int i = 0; i < n; i++) {
        st.insert(a[i]);
    }

    //Find the longest sequence:
    for (auto it : st) {
        //if 'it' is a starting number:
        if (st.find(it - 1) == st.end()) {
            //find consecutive numbers:
            int cnt = 1;
            int x = it;
            while (st.find(x + 1) != st.end()) {
                x = x + 1;
                cnt = cnt + 1;
            }
            longest = max(longest, cnt);
        }
    }
    return longest;
```

```
}

int main()
{
    vector<int> a = {100, 200, 1, 2, 3, 4};
    int ans = longestSuccessiveElements(a);
    cout << "The longest consecutive sequence is " << ans << "\n";
    return 0;
}
```

Output: The longest consecutive sequence is 4.

Time Complexity: O(N) + O(2*N) ~ O(3*N), where N = size of the array.
Reason: O(N) for putting all the elements into the set data structure. After that for every starting element, we are finding the consecutive elements. Though we are using nested loops, the set will be traversed at most twice in the worst case. So, the time complexity is O(2*N) instead of O(N2).

Space Complexity: O(N), as we are using the set data structure to solve this problem.

Note: The time complexity is computed under the assumption that we are using unordered_set and it is taking O(1) for the set operations.

If we consider the worst case the set operations will take O(N) in that case and the total time complexity will be approximately O(N2).
And if we use the set instead of unordered_set, the time complexity for the set operations will be O(logN) and the total time complexity will be O(NlogN).

25. Set Matrix Zero

If any cell `(i, j)` is `0`, then **entire row `i`** and **entire column `j`** must become `0`.

Approach 1: In-place marking with a sentinel (❌ risky)

You mark same row/col by writing `-1` and later turn all `-1` → `0`.

- **Problem:** If the input can already contain `-1` (or any chosen sentinel), you'll corrupt data.

- **Also:** With many zeros, time can degrade to $O(Z \cdot (N+M)) \leq O(NM(N+M))$.
- **Use only** if the matrix domain guarantees the sentinel is safe.

```
#include <bits/stdc++.h>
using namespace std;

void markRow(vector<vector<int>> &matrix, int n, int m, int i) {
    // set all non-zero elements as -1 in the row i:
    for (int j = 0; j < m; j++) {
        if (matrix[i][j] != 0) {
            matrix[i][j] = -1;
        }
    }
}


void markCol(vector<vector<int>> &matrix, int n, int m, int j) {
    // set all non-zero elements as -1 in the col j:
    for (int i = 0; i < n; i++) {
        if (matrix[i][j] != 0) {
            matrix[i][j] = -1;
        }
    }
}

vector<vector<int>> zeroMatrix(vector<vector<int>> &matrix, int n, int m) {

    // Set -1 for rows and cols
    // that contains 0. Don't mark any 0 as -1:

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (matrix[i][j] == 0) {
                markRow(matrix, n, m, i);
                markCol(matrix, n, m, j);
            }
        }
    }

    // Finally, mark all -1 as 0:
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++) {
            if (matrix[i][j] == -1) {
                matrix[i][j] = 0;
            }
        }
    }

    return matrix;
}

int main()
{
    vector<vector<int>> matrix = {{1, 1, 1}, {1, 0, 1}, {1, 1, 1}};
    int n = matrix.size();
    int m = matrix[0].size();
    vector<vector<int>> ans = zeroMatrix(matrix, n, m);

    cout << "The Final matrix is: n";
    for (auto it : ans) {
        for (auto ele : it) {
            cout << ele << " ";
        }
        cout << "n";
    }
    return 0;
}
```

Output: The Final matrix is: 1 0 1
0 0 0
1 0 1

Time Complexity: O((N*M)*(N + M)) + O(N*M), where N = no. of rows in the matrix and M = no. of columns in the matrix.
Reason: Firstly, we are traversing the matrix to find the cells with the value 0. It takes O(N*M). Now, whenever we find any such cell we mark that row and column with -1. This process takes O(N+M). So, combining this the whole process, finding and marking, takes O((N*M)*(N + M)).
Another O(N*M) is taken to mark all the cells with -1 as 0 finally.

Space Complexity: O(1) as we are not using any extra space.

Approach 2: Extra row/col arrays (✅ simple & safe)

Keep two arrays `row[n]`, `col[m]`. First pass marks which rows/cols to zero; second pass zeros them.

- **Time:** `O(NM)` (two passes)
- **Space:** `O(N + M)`

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> zeroMatrix(vector<vector<int>> &matrix, int n, int m) {

    int row[n] = {0}; // row array
    int col[m] = {0}; // col array

    // Traverse the matrix:
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (matrix[i][j] == 0) {
                // mark ith index of row wih 1:
                row[i] = 1;

                // mark jth index of col wih 1:
                col[j] = 1;
            }
        }
    }

    // Finally, mark all (i, j) as 0
    // if row[i] or col[j] is marked with 1.
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (row[i] || col[j]) {
                matrix[i][j] = 0;
            }
        }
    }

    return matrix;
}
```

```
int main()
{
    vector<vector<int>> matrix = {{1, 1, 1}, {1, 0, 1}, {1, 1, 1}};
    int n = matrix.size();
    int m = matrix[0].size();
    vector<vector<int>> ans = zeroMatrix(matrix, n, m);

    cout << "The Final matrix is: n";
    for (auto it : ans) {
        for (auto ele : it) {
            cout << ele << " ";
        }
        cout << "n";
    }
    return 0;
}
```

Output: The Final matrix is: 1 0 1
0 0 0
1 0 1

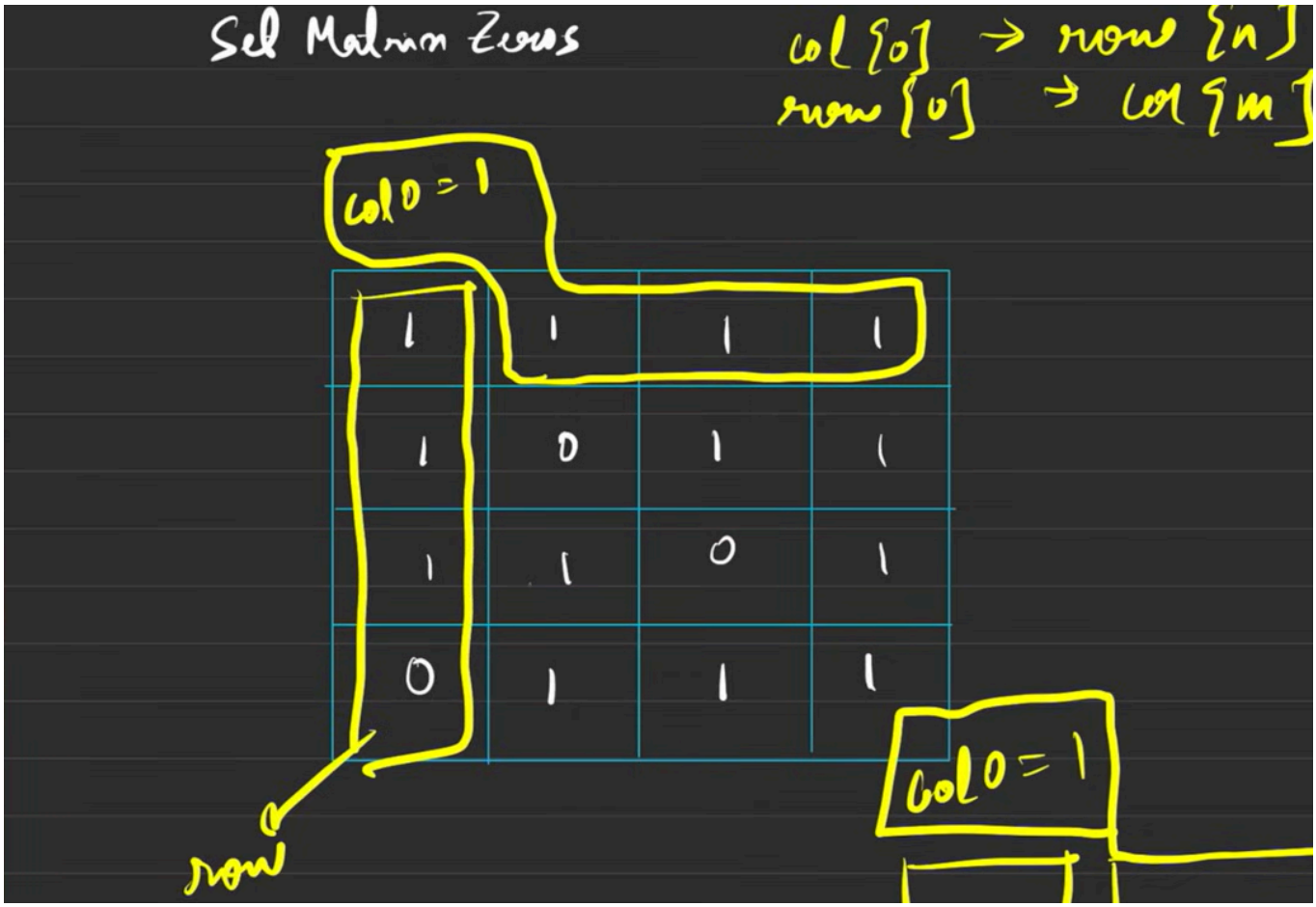
Time Complexity: O(2*(N*M)), where N = no. of rows in the matrix and M = no. of columns in the matrix.
Reason: We are traversing the entire matrix 2 times and each traversal is taking O(N*M) time complexity.

Space Complexity: O(N) + O(M), where N = no. of rows in the matrix and M = no. of columns in the matrix.
Reason: O(N) is for using the row array and O(M) is for using the col array.

Approach 3: O(1) extra space using 1st row & 1st column as markers (✅ optimal)

Use `matrix[i][0]` to mark row `i` and `matrix[0][j]` to mark column `j`.
`matrix[0][0]` is ambiguous (marks both), so track first column separately with `col0`.

- **Time:** `O(NM)` (two passes)
- **Space:** `O(1)`




```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> zeroMatrix(vector<vector<int>> &matrix, int n, int m) {

    // int row[n] = {0}; --> matrix[..][0]
    // int col[m] = {0}; --> matrix[0][..]

    int col0 = 1;
    // step 1: Traverse the matrix and
    // mark 1st row & col accordingly:
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (matrix[i][j] == 0) {
                // mark i-th row:
                matrix[i][0] = 0;

                // mark j-th column:
```

```
            if (j != 0)
                matrix[0][j] = 0;
            else
                col0 = 0;
        }
    }
}

// Step 2: Mark with 0 from (1,1) to (n-1, m-1):
for (int i = 1; i < n; i++) {
    for (int j = 1; j < m; j++) {
        if (matrix[i][j] != 0) {
            // check for col & row:
            if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                matrix[i][j] = 0;
            }
        }
    }
}

//step 3: Finally mark the 1st col & then 1st row:
if (matrix[0][0] == 0) {
    for (int j = 0; j < m; j++) {
        matrix[0][j] = 0;
    }
}
if (col0 == 0) {
    for (int i = 0; i < n; i++) {
        matrix[i][0] = 0;
    }
}

return matrix;
}

int main()
{
    vector<vector<int>> matrix = {{1, 1, 1}, {1, 0, 1}, {1, 1, 1}};
    int n = matrix.size();
    int m = matrix[0].size();
    vector<vector<int>> ans = zeroMatrix(matrix, n, m);

    cout << "The Final matrix is: n";
    for (auto it : ans) {
```

```
        for (auto ele : it) {
            cout << ele << " ";
        }
        cout << "n";
    }
    return 0;
}
```

Output: The Final matrix is: 1 0 1
0 0 0
1 0 1

Time Complexity: O(2*(N*M)), where N = no. of rows in the matrix and M = no. of columns in the matrix.
Reason: In this approach, we are also traversing the entire matrix 2 times and each traversal is taking O(N*M) time complexity.

Space Complexity: O(1) as we are not using any extra space.

26. Rotate Image by 90 degree

Input:

1 2 3
4 5 6
7 8 9

Output (rotated 90° clockwise):

7 4 1
8 5 2
9 6 3

🧩 APPROACH 1 — Using Extra Matrix (Brute Force)

💡 Intuition:

Every element `(i, j)` in the original matrix moves to position `(j, n - 1 - i)` in the rotated matrix.

So:

```
new[j][n - i - 1] = old[i][j];
```

We simply copy into a new matrix based on that relationship.

```
#include<bits/stdc++.h>

using namespace std;
vector < vector < int >> rotate(vector < vector < int >> & matrix) {
    int n = matrix.size();
    vector < vector < int >> rotated(n, vector < int > (n, 0));
    for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        rotated[j][n - i - 1] = matrix[i][j];
    }
    }
    return rotated;
}

int main() {
    vector < vector < int >> arr;
    arr =  {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    vector < vector < int >> rotated = rotate(arr);
    cout << "Rotated Image" << endl;
    for (int i = 0; i < rotated.size(); i++) {
    for (int j = 0; j < rotated[0].size(); j++) {
        cout << rotated[i][j] << " ";
    }
    cout << "n";
    }

}
```

Output:

Rotated Image
7 4 1
8 5 2
9 6 3

Time Complexity: O(N*N) to linearly iterate and put it into some other matrix.

Space Complexity: O(N*N) to copy it into some other matrix.

⚡ APPROACH 2 — In-place (Optimal)

💡 Intuition:

We can do it **without using extra space**.

We know:

90° rotation = **Transpose** + **Reverse every row**

Why it works:

Transpose: converts rows → columns
(matrix[i][j] ↔ matrix[j][i])

```
 1  2  3        1  4  7
4  5  6   →    2  5  8
7  8  9        3  6  9
```

Reverse each row: rotates the matrix

```
 1  4  7        7  4  1
2  5  8   →    8  5  2
3  6  9        9  6  3
```

```
#include<bits/stdc++.h>

using namespace std;
void rotate(vector < vector < int >> & matrix) {
    int n = matrix.size();
```

```
    //transposing the matrix
    for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        swap(matrix[i][j], matrix[j][i]);
    }
    }
    //reversing each row of the matrix
    for (int i = 0; i < n; i++) {
    reverse(matrix[i].begin(), matrix[i].end());
    }
}

int main() {
    vector < vector < int >> arr;
    arr =  {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    rotate(arr);
    cout << "Rotated Image" << endl;
    for (int i = 0; i < arr.size(); i++) {
    for (int j = 0; j < arr[0].size(); j++) {
        cout << arr[i][j] << " ";
    }
    cout << "n";
    }

}
```

Output:

```
Rotated Image
7 4 1
8 5 2
9 6 3
```

Time Complexity: O(N*N) + O(N*N).One O(N*N) is for transposing the matrix and the other is for reversing the matrix.

Space Complexity: O(1).

27. Spiral Traversal of Matrix

🎯 Problem Statement

Given an `n × m` matrix, print all its elements in **spiral order traversal** (clockwise).

```
1  2  3  4

5  6  7  8

9 10 11 12

13 14 15 16

Output:

1 2 3 4 8 12 16 15 14 13 9 5 6 7 11 10
```

🧠 Intuition

Iimagine the traversal in **layers (rings)**:

1. Traverse the **top row** (→)
2. Traverse the **right column** (↓)
3. Traverse the **bottom row** (←)
4. Traverse the **left column** (↑)

After each full cycle, we move **one layer inward** by updating:

- `top++`
- `right--`
- `bottom--`
- `left++`

We continue until all layers are covered (`top <= bottom && left <= right`).

```
#include <bits/stdc++.h>

using namespace std;

vector<int> printSpiral(vector<vector<int>> mat) {

  // Define ans array to store the result.
  vector<int> ans;

  int n = mat.size(); // no. of nows
  int m = mat[0].size(); // no. of columns

  // Initialize the pointers reqd for traversal.
  int top = 0, left = 0, bottom = n - 1, right = m - 1;

  // Loop until all elements are not traversed.
  while (top <= bottom && left <= right) {

    // For moving left to right
    for (int i = left; i <= right; i++)
      ans.push_back(mat[top][i]);

    top++;

    // For moving top to bottom.
    for (int i = top; i <= bottom; i++)
      ans.push_back(mat[i][right]);

    right--;

    // For moving right to left.
    if (top <= bottom) {
      for (int i = right; i >= left; i--)
       ans.push_back(mat[bottom][i]);

      bottom--;
    }

    // For moving bottom to top.
    if (left <= right) {
      for (int i = bottom; i >= top; i--)
        ans.push_back(mat[i][left]);

      left++;
```

```
    }
  }
  return ans;
}

int main() {

  //Matrix initialization.
  vector<vector<int>> mat   {{1, 2, 3, 4},
                             {5, 6, 7, 8},
                                {9, 10, 11, 12},
                                    {13, 14, 15, 16}};

  vector<int> ans = printSpiral(mat);

  for(int i = 0;i<ans.size();i++){

      cout<<ans[i]<<" ";
  }

  cout<<endl;

  return 0;
}
```

Output:

1 2 3 4 8 12 16 15 14 13 9 5 6 7 11 10

Time Complexity: O(m x n) { Since all the elements are being traversed once and there are total n x m elements (m elements in each row and total n rows) so the time complexity will be O(n x m)}.

Space Complexity: O(n) { Extra Space used for storing traversal in the ans array }.

28. Count Subarray sum Equals K

1. **Brute force `O(N^3)`**

Har subarray `[i..j]` ka sum inner loop se nikaalo. (Slow)

```
#include <bits/stdc++.h>
using namespace std;

int findAllSubarraysWithGivenSum(vector < int > & arr, int k) {
    int n = arr.size(); // size of the given array.
    int cnt = 0; // Number of subarrays:

    for (int i = 0 ; i < n; i++) { // starting index i
        for (int j = i; j < n; j++) { // ending index j

            // calculate the sum of subarray [i...j]
            int sum = 0;
            for (int K = i; K <= j; K++)
                sum += arr[K];

            // Increase the count if sum == k:
            if (sum == k)
                cnt++;
        }
    }
    return cnt;
}

int main()
{
    vector arr = {3, 1, 2, 4};
    int k = 6;
    int cnt = findAllSubarraysWithGivenSum(arr, k);
    cout << "The number of subarrays is: " << cnt << "\n";
    return 0;
}
```

Output: The number of subarrays is: 2

Time Complexity: O(N3), where N = size of the array.

Reason: We are using three nested loops here. Though all are not running for exactly N times, the time complexity will be approximately O(N3).

Space Complexity: O(1) as we are not using any extra space.

2. **Better `O(N^2)`**

Har i se start karke running sum rakhte jao: `sum += arr[j]`. (Still quadratic)

```
#include <bits/stdc++.h>
using namespace std;

int findAllSubarraysWithGivenSum(vector < int > & arr, int k) {
    int n = arr.size(); // size of the given array.
    int cnt = 0; // Number of subarrays:

    for (int i = 0 ; i < n; i++) { // starting index i
        int sum = 0;
        for (int j = i; j < n; j++) { // ending index j
            // calculate the sum of subarray [i...j]
            // sum of [i..j-1] + arr[j]
            sum += arr[j];

            // Increase the count if sum == k:
            if (sum == k)
                cnt++;
        }
    }
    return cnt;
}

int main()
{
    vector arr = {3, 1, 2, 4};
    int k = 6;
    int cnt = findAllSubarraysWithGivenSum(arr, k);
    cout << "The number of subarrays is: " << cnt << "\n";
    return 0;
}
```

Output: The number of subarrays is: 2

Time Complexity: O(N2), where N = size of the array.
Reason: We are using two nested loops here. As each of them is running for exactly N times, the time complexity will be approximately O(N2).

Space Complexity: O(1) as we are not using any extra space.

3. **Optimal (Prefix-sum + Hash) `O(N)` avg**

Idea: agar `preSum` current tak ka sum hai, to jitni baar **`preSum - K`** pehle aa chuka hai, utni subarrays end-at-i ka sum K banate hain.
Isliye ek map me **prefix-sum frequency** rakhte hain. Start me `mpp[0] = 1` (empty prefix) zaroori hai.

Note: Two-pointer sliding window **sirf positive numbers** pe kaam karta; is problem me negatives ho sakte hain, to prefix-sum + hash hi best.

```
#include <bits/stdc++.h>
using namespace std;

long long countSubarraysWithSumK(const vector<int>& arr, long long k) {
    unordered_map<long long, long long> freq; // prefix-sum -> count
    long long preSum = 0, cnt = 0;

    freq[0] = 1; // empty prefix

    for (int x : arr) {
        preSum += x;
        long long need = preSum - k;   // we want previous prefix = preSum - k
        if (freq.find(need) != freq.end())
            cnt += freq[need];
        freq[preSum]++; // record current prefix
    }
    return cnt;
}

int main() {
    vector<int> arr = {3, 1, 2, 4};
    long long k = 6;
    cout << "The number of subarrays is: " << countSubarraysWithSumK(arr, k) << "\n";
    return 0;
}
```

Output: The number of subarrays is: 2
Time Complexity: O(N) or O(N*logN) depending on which map data structure we are using, where N = size of the array.
Reason: For example, if we are using an unordered_map data structure in C++ the time complexity will be O(N) but if we are using a map data structure, the time complexity will be O(N*logN). The least complexity will be O(N) as we are using a loop to traverse the array.

Space Complexity: O(N) as we are using a map data structure.

Kab `long long`?

Prefix sums overflow avoid karne ke liye (bade values), `preSum` aur `k` ko `long long` rakhna safe rehta hai.

29. Program to generate Pascal's Triangle

Pascal's Triangle is a triangular arrangement of binomial coefficients.

```
     1
    1 1
   1 2 1
  1 3 3 1
 1 4 6 4 1
```

Formally:

```
arr[i][j] = arr[i-1][j-1] + arr[i-1][j]
```
with `arr[i][0] = arr[i][i] = 1`

🧩 Approach 1 — Generate Full Pascal's Triangle (Brute Force)

We generate all rows one by one using the rule:

```
row[j] = prevRow[j-1] + prevRow[j]
```

#include <bits/stdc++.h>
using namespace std;

```
// Class containing Pascal's Triangle generation logic
class Solution {
public:
    // Function to generate Pascal's Triangle up to numRows
    vector<vector<int>> generate(int numRows) {
        // Result vector to hold all rows
        vector<vector<int>> triangle;

        // Loop for each row
        for (int i = 0; i < numRows; i++) {
            // Create a row with size (i+1) and initialize all elements to 1
            vector<int> row(i + 1, 1);

            // Fill elements from index 1 to i-1 (middle values)
            for (int j = 1; j < i; j++) {
                // Each element = sum of two elements above it
                row[j] = triangle[i - 1][j - 1] + triangle[i - 1][j];
            }

            // Add current row to the triangle
            triangle.push_back(row);
        }
        return triangle;
    }
};

int main() {
    Solution obj;
    int n = 5;

    // Generate and print Pascal's Triangle
    vector<vector<int>> result = obj.generate(n);
    for (auto &row : result) {
        for (auto &val : row) cout << val << " ";
        cout << endl;
    }
}
```

Time Complexity: O(N^2), we generate all the elements in first N rows sequentially one by one.
Space Complexity: O(N^2), additional space used for storing the entire pascal triangle.

🧩 Approach 2 — Generate Only Nth Row (Efficient)

We can compute the **Nth row directly** using the **binomial formula**:

C(n,k) = C(n,k-1) * (n-k)/k

This lets us compute the entire row in O(N) time without using the triangle.

```
#include <bits/stdc++.h>
using namespace std;

// Class containing Pascal's Triangle row generation logic
class Solution {
public:
    // Function to generate the Nth row of Pascal's Triangle
    vector<long long> getNthRow(int N) {
        // Result vector to store the row
        vector<long long> row;

        // First value of the row is always 1
        long long val = 1;
        row.push_back(val);

        // Compute remaining values using the relation:
        // C(n, k) = C(n, k-1) * (n-k) / k
        for (int k = 1; k < N; k++) {
            val = val * (N - k) / k;
            row.push_back(val);
        }

        return row;
    }
};

int main() {
    int N = 5; // Example: 5th row
    Solution sol;
    vector<long long> result = sol.getNthRow(N);

    // Print the row
    for (auto num : result) {
        cout << num << " ";
    }
    return 0;
```

}

Time Complexity: O(N), we iterate N times to compute each element of the row in O(1) time using the direct relation.
Space Complexity: O(N), additional space used for storing the Nth row.

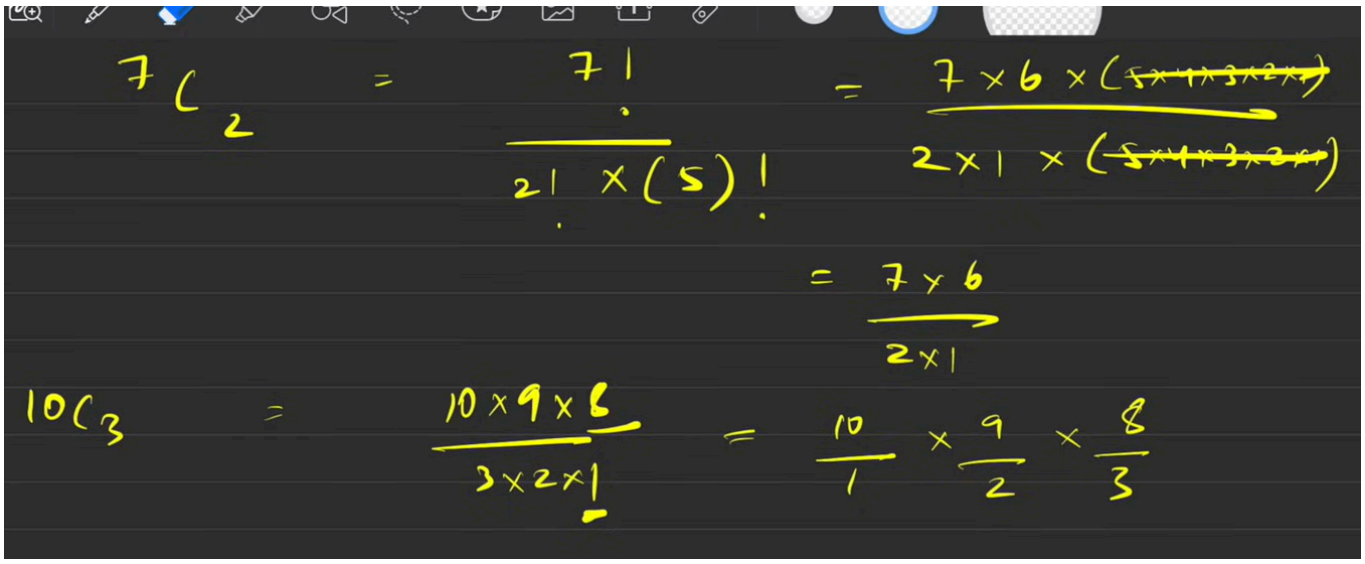
🧩 Approach 3 — Find the Element at (r, c)

The element in the r-th row and c-th column of Pascal's triangle is:

Element(r,c)= C(r−1,c−1)

We can calculate this using the iterative formula for binomial coefficients:

C(n,k) = n! / ((n-k)! * k!




```
#include <bits/stdc++.h>
using namespace std;

// Solution class to find the (r, c) element of Pascal's Triangle
class Solution {
public:
    // Function to compute binomial coefficient (nCr)
```

```
    long long findPascalElement(int r, int c) {
        // Element is C(r-1, c-1)
        int n = r - 1;
        int k = c - 1;

        long long result = 1;

        // Compute C(n, k) using iterative formula
        for (int i = 0; i < k; i++) {
            result *= (n - i);
            result /= (i + 1);
        }

        return result;
    }
};

int main() {
    Solution sol;
    int r = 5, c = 3;
    cout << sol.findPascalElement(r, c);
    return 0;
}
```

Time Complexity: O(min(c,r−c)), The loop runs for min(c−1,r−c) iterations because binomial coefficients are symmetric.
Space Complexity: O(1), constant additional space is used.

30. Majority Elements(>N/3 times) | Find the elements that appears more than N/3 times in the array

1) Brute force ($O(N^2)$, $O(1)$)

```
#include <bits/stdc++.h>
using namespace std;

vector<int> majorityElement(vector<int> v) {
    int n = v.size(); //size of the array
    vector<int> ls; // list of answers

    for (int i = 0; i < n; i++) {
        //selected element is v[i]:
        // Checking if v[i] is not already
        // a part of the answer:
        if (ls.size() == 0 || ls[0] != v[i]) {
            int cnt = 0;
            for (int j = 0; j < n; j++) {
                // counting the frequency of v[i]
                if (v[j] == v[i]) {
                    cnt++;
                }
            }

            // check if frquency is greater than n/3:
            if (cnt > (n / 3))
                ls.push_back(v[i]);
        }

        if (ls.size() == 2) break;
    }

    return ls;
}

int main()
{
    vector<int> arr = {11, 33, 33, 11, 33, 11};
    vector<int> ans = majorityElement(arr);
    cout << "The majority elements are: ";
    for (auto it : ans)
```

```
        cout << it << " ";
    cout << "\n";
    return 0;
}
```

Output: The majority elements are: 11 33

Time Complexity: O(N2), where N = size of the given array.
Reason: For every element of the array the inner loop runs for N times. And there are N elements in the array. So, the total time complexity is O(N2).

Space Complexity: O(1) as we are using a list that stores a maximum of 2 elements. The space used is so small that it can be considered constant.

2) Hash map / unordered_map (O(N logN) or O(N), O(N))

Your map version is correct; note that we can push when count becomes `== floor(N/3)+1`. Using `unordered_map` is typically O(N):

```
#include <bits/stdc++.h>
using namespace std;

vector<int> majorityElement(vector<int> v) {
    int n = v.size(); //size of the array
    vector<int> ls; // list of answers

    //declaring a map:
    map<int, int> mpp;

    // least occurrence of the majority element:
    int mini = int(n / 3) + 1;

    //storing the elements with its occurnce:
    for (int i = 0; i < n; i++) {
        mpp[v[i]]++;

        //checking if v[i] is
```

```
        // the majority element:
        if (mpp[v[i]] == mini) {
            ls.push_back(v[i]);
        }
        if (ls.size() == 2) break;
    }

    return ls;
}

int main()
{
    vector<int> arr = {11, 33, 33, 11, 33, 11};
    vector<int> ans = majorityElement(arr);
    cout << "The majority elements are: ";
    for (auto it : ans)
        cout << it << " ";
    cout << "\n";
    return 0;
}
```

Output: The majority elements are: 33 11

Time Complexity: O(N*logN), where N = size of the given array.
Reason: We are using a map data structure. Insertion in the map takes logN time. And we are doing it for N elements. So, it results in the first term O(N*logN).
If we use unordered_map instead, the first term will be O(N) for the best and average case and for the worst case, it will be O(N2).

Space Complexity: O(N) as we are using a map data structure. We are also using a list that stores a maximum of 2 elements. That space used is so small that it can be considered constant.

3) Extended Boyer–Moore (Best: O(N), O(1))

This tracks up to **two candidates** and their counts, then **verifies** them.;

```
#include <bits/stdc++.h>
using namespace std;

vector<int> majorityElement_boyerMoore(const vector<int>& v) {
    int n = v.size();
    int cnt1 = 0, cnt2 = 0;
```

```
    int el1 = 0,  el2 = 0;

    // 1) Find potential candidates
    for (int x : v) {
        if (cnt1 > 0 && x == el1) cnt1++;
        else if (cnt2 > 0 && x == el2) cnt2++;
        else if (cnt1 == 0) { el1 = x; cnt1 = 1; }
        else if (cnt2 == 0) { el2 = x; cnt2 = 1; }
        else { cnt1--; cnt2--; }
    }

    // 2) Verify counts
    cnt1 = cnt2 = 0;
    for (int x : v) {
        if (x == el1) cnt1++;
        else if (x == el2) cnt2++;
    }

    vector<int> ans;
    if (cnt1 > n/3) ans.push_back(el1);
    if (cnt2 > n/3) ans.push_back(el2);
    return ans;
}
```

Output: The majority elements are: 11 33
Time Complexity: O(N) + O(N), where N = size of the given array.
Reason: The first O(N) is to calculate the counts and find the expected majority elements. The second one is to check if the calculated elements are the majority ones or not.

Space Complexity: O(1) as we are only using a list that stores a maximum of 2 elements. The space used is so small that it can be considered constant.

31. 3 Sum : Find triplets that add up to a zero

1) Brute force ($O(N^3)$)
Try every (i, j, k) with i<j<k.

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> triplet(int n, vector<int> &arr) {
    set<vector<int>> st;

    // check all possible triplets:
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            for (int k = j + 1; k < n; k++) {
                if (arr[i] + arr[j] + arr[k] == 0) {
                    vector<int> temp = {arr[i], arr[j], arr[k]};
                    sort(temp.begin(), temp.end());
                    st.insert(temp);
                }
            }
        }
    }

    //store the set elements in the answer:
    vector<vector<int>> ans(st.begin(), st.end());
    return ans;
}


int main()
{
    vector<int> arr = { -1, 0, 1, 2, -1, -4};
    int n = arr.size();
    vector<vector<int>> ans = triplet(n, arr);
    for (auto it : ans) {
        cout << "[";
        for (auto i : it) {
            cout << i << " ";
        }
        cout << "] ";
    }
```

```
    cout << "\n";
    return 0;
}
```

Output: [-1 -1 2] [-1 0 1]

Time Complexity: O(N3 * log(no. of unique triplets)), where N = size of the array.
Reason: Here, we are mainly using 3 nested loops. And inserting triplets into the set takes O(log(no. of unique triplets)) time complexity. But we are not considering the time complexity of sorting as we are just sorting 3 elements every time.

Space Complexity: O(2 * no. of the unique triplets) as we are using a set data structure and a list to store the triplets.

2) "Two loops + hash set" (O(N²))

Fix `i`, then for the subarray `(i+1 … n-1)` you're basically doing **2-sum** to target `-arr[i]`.
Keep a `set<int>` (or `unordered_set<int>`) while moving `j`.
If `third = -(arr[i]+arr[j])` exists in the set, you found a triplet `{arr[i], arr[j], third}`.
Still use a `set<vector<int>>` to dedupe final triplets (because order/duplicates can repeat).

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> triplet(int n, vector<int> &arr) {
    set<vector<int>> st;

    for (int i = 0; i < n; i++) {
        set<int> hashset;
        for (int j = i + 1; j < n; j++) {
            //Calculate the 3rd element:
            int third = -(arr[i] + arr[j]);

            //Find the element in the set:
            if (hashset.find(third) != hashset.end()) {
                vector<int> temp = {arr[i], arr[j], third};
                sort(temp.begin(), temp.end());
                st.insert(temp);
            }
```

```
            hashset.insert(arr[j]);
        }
    }

    //store the set in the answer:
    vector<vector<int>> ans(st.begin(), st.end());
    return ans;
}


int main()
{
    vector<int> arr = { -1, 0, 1, 2, -1, -4};
    int n = arr.size();
    vector<vector<int>> ans = triplet(n, arr);
    for (auto it : ans) {
        cout << "[";
        for (auto i : it) {
            cout << i << " ";
        }
        cout << "] ";
    }
    cout << "\n";
    return 0;
}
```

Output: [-1 -1 2] [-1 0 1]

Time Complexity: O(N2 * log(no. of unique triplets)), where N = size of the array.
Reason: Here, we are mainly using 3 nested loops. And inserting triplets into the set takes O(log(no. of unique triplets)) time complexity. But we are not considering the time complexity of sorting as we are just sorting 3 elements every time.

Space Complexity: O(2 * no. of the unique triplets) + O(N) as we are using a set data structure and a list to store the triplets and extra O(N) for storing the array elements in another set.

3) Sort + two pointers (optimal O(N²), in-place dedupe)
Sort the array once (O(N log N)).
Loop `i` from 0..n-1 (skip duplicates for `i`).
For each `i`, run a classic two-pointer 2-sum on the subarray `(i+1 … n-1)`:

- `j = i+1`, `k = n-1`
- If `arr[i]+arr[j]+arr[k] < 0` → `j++`

- If `> 0` → `k--`
- If `== 0` → record triplet, then move `j++` and `k--` **and skip duplicates** for both sides.

Why it's best: O(N²) time, **O(1) extra space**, and duplicate-skipping happens on the fly, so no `set<vector<int>>` needed.

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> triplet(int n, vector<int> &arr) {
    vector<vector<int>> ans;
    sort(arr.begin(), arr.end());
    for (int i = 0; i < n; i++) {
        //remove duplicates:
        if (i != 0 && arr[i] == arr[i - 1]) continue;

        //moving 2 pointers:
        int j = i + 1;
        int k = n - 1;
        while (j < k) {
            int sum = arr[i] + arr[j] + arr[k];
            if (sum < 0) {
                j++;
            }
            else if (sum > 0) {
                k--;
            }
            else {
                vector<int> temp = {arr[i], arr[j], arr[k]};
                ans.push_back(temp);
                j++;
                k--;
                //skip the duplicates:
                while (j < k && arr[j] == arr[j - 1]) j++;
                while (j < k && arr[k] == arr[k + 1]) k--;
            }
        }
    }
    return ans;
}


int main()
```

```
{
    vector<int> arr = { -1, 0, 1, 2, -1, -4};
    int n = arr.size();
    vector<vector<int>> ans = triplet(n, arr);
    for (auto it : ans) {
        cout << "[";
        for (auto i : it) {
            cout << i << " ";
        }
        cout << "] ";
    }
    cout << "\n";
    return 0;
}
```

Output: [-1 -1 2] [-1 0 1]

Time Complexity: O(NlogN)+O(N2), where N = size of the array.
Reason: The pointer i, is running for approximately N times. And both the pointers j and k combined can run for approximately N times including the operation of skipping duplicates. So the total time complexity will be O(N2).

Space Complexity: O(no. of quadruplets), This space is only used to store the answer. We are not using any extra space to solve this problem. So, from that perspective, space complexity can be written as O(1).

32. 4 Sum | Find Quads that add up to a target value

1) Pure brute force (4 nested loops + set) — O(N^4)

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> fourSum(vector<int>& nums, int target) {
    int n = nums.size(); //size of the array
    set<vector<int>> st;

    //checking all possible quadruplets:
    for (int i = 0; i < n; i++) {
```

```
        for (int j = i + 1; j < n; j++) {
            for (int k = j + 1; k < n; k++) {
                for (int l = k + 1; l < n; l++) {
                    // taking bigger data type
                    // to avoid integer overflow:
                    long long sum = nums[i] + nums[j];
                    sum += nums[k];
                    sum += nums[l];

                    if (sum == target) {
                        vector<int> temp = {nums[i], nums[j], nums[k], nums[l]};
                        sort(temp.begin(), temp.end());
                        st.insert(temp);
                    }
                }
            }
        }
    }
    vector<vector<int>> ans(st.begin(), st.end());
    return ans;
}

int main()
{
    vector<int> nums = {4, 3, 3, 4, 4, 2, 1, 2, 1, 1};
    int target = 9;
    vector<vector<int>> ans = fourSum(nums, target);
    cout << "The quadruplets are: \n";
    for (auto it : ans) {
        cout << "[";
        for (auto ele : it) {
            cout << ele << " ";
        }
        cout << "] ";
    }
    cout << "\n";
    return 0;
}
```

Output: The quadruplets are: [1 1 3 4] [1 2 2 4] [1 2 3 3]

Time Complexity: O(N4), where N = size of the array.

Reason: Here, we are mainly using 4 nested loops. But we not considering the time complexity of sorting as we are just sorting 4 elements every time.

Space Complexity: O(2 * no. of the quadruplets) as we are using a set data structure and a list to store the quads.

2) 3 loops + hash set — $O(N^3)$ avg

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> fourSum(vector<int>& nums, int target) {
    int n = nums.size(); //size of the array
    set<vector<int>> st;

    //checking all possible quadruplets:
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            set<long long> hashset;
            for (int k = j + 1; k < n; k++) {
                // taking bigger data type
                // to avoid integer overflow:
                long long sum = nums[i] + nums[j];
                sum += nums[k];
                long long fourth = target - sum;
                if (hashset.find(fourth) != hashset.end()) {
                    vector<int> temp = {nums[i], nums[j], nums[k], (int)(fourth)};
                    sort(temp.begin(), temp.end());
                    st.insert(temp);
                }
                // put the kth element into the hashset:
                hashset.insert(nums[k]);
            }
        }
    }
    vector<vector<int>> ans(st.begin(), st.end());
    return ans;
}

int main()
{
```


Prepared by Vinay Kajla

Instagram: @Vinay.Kajla_

```
    vector<int> nums = {4, 3, 3, 4, 4, 2, 1, 2, 1, 1};
    int target = 9;
    vector<vector<int>> ans = fourSum(nums, target);
    cout << "The quadruplets are: \n";
    for (auto it : ans) {
        cout << "[";
        for (auto ele : it) {
            cout << ele << " ";
        }
        cout << "] ";
    }
    cout << "\n";
    return 0;
}
```

Output: The quadruplets are: [1 1 3 4] [1 2 2 4] [1 2 3 3]
Time Complexity: O(N3*log(M)), where N = size of the array, M = no. of elements in the set.
Reason: Here, we are mainly using 3 nested loops, and inside the loops there are some operations on the set data structure which take log(M) time complexity.

Space Complexity: O(2 * no. of the quadruplets)+O(N)
Reason: we are using a set data structure and a list to store the quads. This results in the first term. And the second space is taken by the set data structure we are using to store the array elements. At most, the set can contain approximately all the array elements and so the space complexity is O(N).

3) Sort + two pointers (optimal) — O(N³), O(1) extra

- Sort the array.
- Fix `i` (skip duplicates), fix `j` (skip duplicates).
- Two pointers `k=j+1`, `l=n-1`:
    - If sum < target → `k++`
    - If sum > target → `l--`
    - If sum == target → push quad, then `k++`, `l--` and **skip duplicates** on both sides.
- **Why it's best:** tight O(N³), no sets, duplicate handling is clean and deterministic.

```
#include <bits/stdc++.h>
using namespace std;
```

```
vector<vector<int>> fourSum(vector<int>& nums, int target) {
    int n = nums.size(); //size of the array
    vector<vector<int>> ans;

    // sort the given array:
    sort(nums.begin(), nums.end());

    //calculating the quadruplets:
    for (int i = 0; i < n; i++) {
        // avoid the duplicates while moving i:
        if (i > 0 && nums[i] == nums[i - 1]) continue;
        for (int j = i + 1; j < n; j++) {
            // avoid the duplicates while moving j:
            if (j > i + 1 && nums[j] == nums[j - 1]) continue;

            // 2 pointers:
            int k = j + 1;
            int l = n - 1;
            while (k < l) {
                long long sum = nums[i];
                sum += nums[j];
                sum += nums[k];
                sum += nums[l];
                if (sum == target) {
                    vector<int> temp = {nums[i], nums[j], nums[k], nums[l]};
                    ans.push_back(temp);
                    k++; l--;

                    //skip the duplicates:
                    while (k < l && nums[k] == nums[k - 1]) k++;
                    while (k < l && nums[l] == nums[l + 1]) l--;
                }
                else if (sum < target) k++;
                else l--;
            }
        }
    }

    return ans;
}

int main()
{
    vector<int> nums = {4, 3, 3, 4, 4, 2, 1, 2, 1, 1};
```

```
    int target = 9;
    vector<vector<int>> ans = fourSum(nums, target);
    cout << "The quadruplets are: \n";
    for (auto it : ans) {
        cout << "[";
        for (auto ele : it) {
            cout << ele << " ";
        }
        cout << "] ";
    }
    cout << "\n";
    return 0;
}
```

Output:The quadruplets are: [1 1 3 4] [1 2 2 4] [1 2 3 3]
Time Complexity: O(N3), where N = size of the array.
Reason: Each of the pointers i and j, is running for approximately N times. And both the pointers k and l combined can run for approximately N times including the operation of skipping duplicates. So the total time complexity will be O(N3).

Space Complexity: O(no. of quadruplets), This space is only used to store the answer. We are not using any extra space to solve this problem. So, from that perspective, space complexity can be written as O(1).

33. Length of the longest subarray with zero Sum

Input: [9, -3, 3, -1, 6, -5]
Output: 5
Explanation:
Subarray [-3, 3, -1, 6, -5] ka sum = 0

🧩 Brute Force Approach

🔹 Logic:

1. Har index `i` se start karte hue har possible `j` tak sum nikalo.
2. Agar kabhi `sum == 0` ho jaye, to `length = j - i + 1` calculate karo.
3. `maxLen` me maximum length store kar lo.

```
#include <bits/stdc++.h>
using namespace std;

int longestSubarrayBrute(vector<int> &a) {
    int n = a.size();
    int maxLen = 0;

    for (int i = 0; i < n; i++) {
        int sum = 0;
        for (int j = i; j < n; j++) {
            sum += a[j];
            if (sum == 0) {
                maxLen = max(maxLen, j - i + 1);
            }
        }
    }
    return maxLen;
}

int main() {
    vector<int> a = {9, -3, 3, -1, 6, -5};
    cout << longestSubarrayBrute(a);
}
```

Time Complexity: O(N^2) as we have two loops for traversal

Space Complexity: O(1) as we aren't using any extra space.

Can this be done in a single traversal? Let's check :)

🚀 Optimized Approach (Using HashMap)

- **Idea:**

Yahan ek smart trick hai:

Agar kisi prefix sum (sum from 0 to i) ko humne pehle bhi dekha hai,
to iska matlab hai ki dono occurrences ke beech ka subarray ka sum = 0 hai!

🔹 Step by Step:

1. Maintain a variable `sum = 0` (prefix sum).
2. Traverse array ke har element par:
   - `sum += A[i]`
   - Agar `sum == 0`, matlab start se i tak sum 0 hua ⇒ `maxi = i + 1`
   - Agar `sum` pehle bhi map me mila, matlab pehle se koi prefix sum same tha.
     ⇒ beech ka portion ka sum zero hoga.
     Length = `i - mpp[sum]`
     `maxi = max(maxi, i - mpp[sum])`
   - Agar `sum` pehli baar aaya hai ⇒ `mpp[sum] = i` store kar lo.
3. End me `maxi` return karo.

```
#include <bits/stdc++.h>
using namespace std;

int maxLen(int A[], int n) {
    unordered_map<int, int> mpp;
    int maxi = 0;
    int sum = 0;

    for (int i = 0; i < n; i++) {
        sum += A[i];

        if (sum == 0) {
            maxi = i + 1;
        } else {
            if (mpp.find(sum) != mpp.end()) {
                maxi = max(maxi, i - mpp[sum]);
            } else {
                mpp[sum] = i;
            }
        }
    }
    return maxi;
}

int main() {
```

```
    int A[] = {9, -3, 3, -1, 6, -5};
    int n = sizeof(A) / sizeof(A[0]);
    cout << maxLen(A, n);
}
```

Time Complexity: O(N), as we are traversing the array only once

Space Complexity: O(N), in the worst case we would insert all array elements prefix sum into our hashmap

34. Count the number of subarrays with given xor K

Array diya hai (0/positive/negative sab ho sakte). Hume **continuous subarrays** ki count chahiye jinka **XOR = K** ho.

Example:
`a = [4, 2, 2, 6, 4], K = 6` → **answer = 4**

Wo 4 subarrays:

1. `[4,2]` (4^2 = 6)
2. `[4,2,2,6,4]` index 0..4 ka kuch part? — actually correct 4 are:
    - `[4,2]` (0..1)
    - `[2,2,6]` (1..3)
    - `[6]` (3..3)
    - `[2,6,4]` (2..4)

🧩 Approach 1: Brute Force (O(N³))

Idea: har `(i,j)` subarray banao, uska XOR ek aur loop se nikalo, agar `== K` to count++.

```
#include <bits/stdc++.h>
using namespace std;
```

```
int subarraysWithXorK(vector<int> a, int k) {
    int n = a.size(); //size of the given array.
    int cnt = 0;

    // Step 1: Generating subarrays:
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {

            //step 2:calculate XOR of all
            // elements:
            int xorr = 0;
            for (int K = i; K <= j; K++) {
                xorr = xorr ^ a[K];
            }

            // step 3:check XOR and count:
            if (xorr == k) cnt++;
        }
    }
    return cnt;
}

int main()
{
    vector<int> a = {4, 2, 2, 6, 4};
    int k = 6;
    int ans = subarraysWithXorK(a, k);
    cout << "The number of subarrays with XOR k is: "
         << ans << "\n";
    return 0;
}
```

Output: The number of subarrays with XOR k is: 4

Time Complexity: O(N3) approx., where N = size of the array.
Reason: We are using three nested loops, each running approximately N times.

Space Complexity: O(1) as we are not using any extra space.

⚡ Approach 2: Carry-forward XOR (O(N²))

Observation: Agar hum `i` fix kar dein, to `xr = 0` se start karke `j` badhate jao aur `xr ^= a[j]` rakhte jao. Har step pe check karo `xr==K`.

```
#include <bits/stdc++.h>
using namespace std;

int subarraysWithXorK(vector<int> a, int k) {
    int n = a.size(); //size of the given array.
    int cnt = 0;

    // Step 1: Generating subarrays:
    for (int i = 0; i < n; i++) {
        int xorr = 0;
        for (int j = i; j < n; j++) {

            //step 2:calculate XOR of all
            // elements:
            xorr = xorr ^ a[j];

            // step 3:check XOR and count:
            if (xorr == k) cnt++;
        }
    }
    return cnt;
}

int main()
{
    vector<int> a = {4, 2, 2, 6, 4};
    int k = 6;
    int ans = subarraysWithXorK(a, k);
    cout << "The number of subarrays with XOR k is: "
         << ans << "\n";
    return 0;
}
```

Output: The number of subarrays with XOR k is: 4

Complexity Analysis

Time Complexity: O(N2), where N = size of the array.
Reason: We are using two nested loops here. As each of them is running for N times, the time complexity will be approximately O(N2).

Space Complexity: O(1) as we are not using any extra space.

🚀 Approach 3: Optimal – Prefix XOR + HashMap (O(N))

Core intuition

Prefix XOR define karo: `pref[i] = a[0] ^ a[1] ^ ... ^ a[i]`

Kisi subarray `(l..r)` ka XOR hota hai:

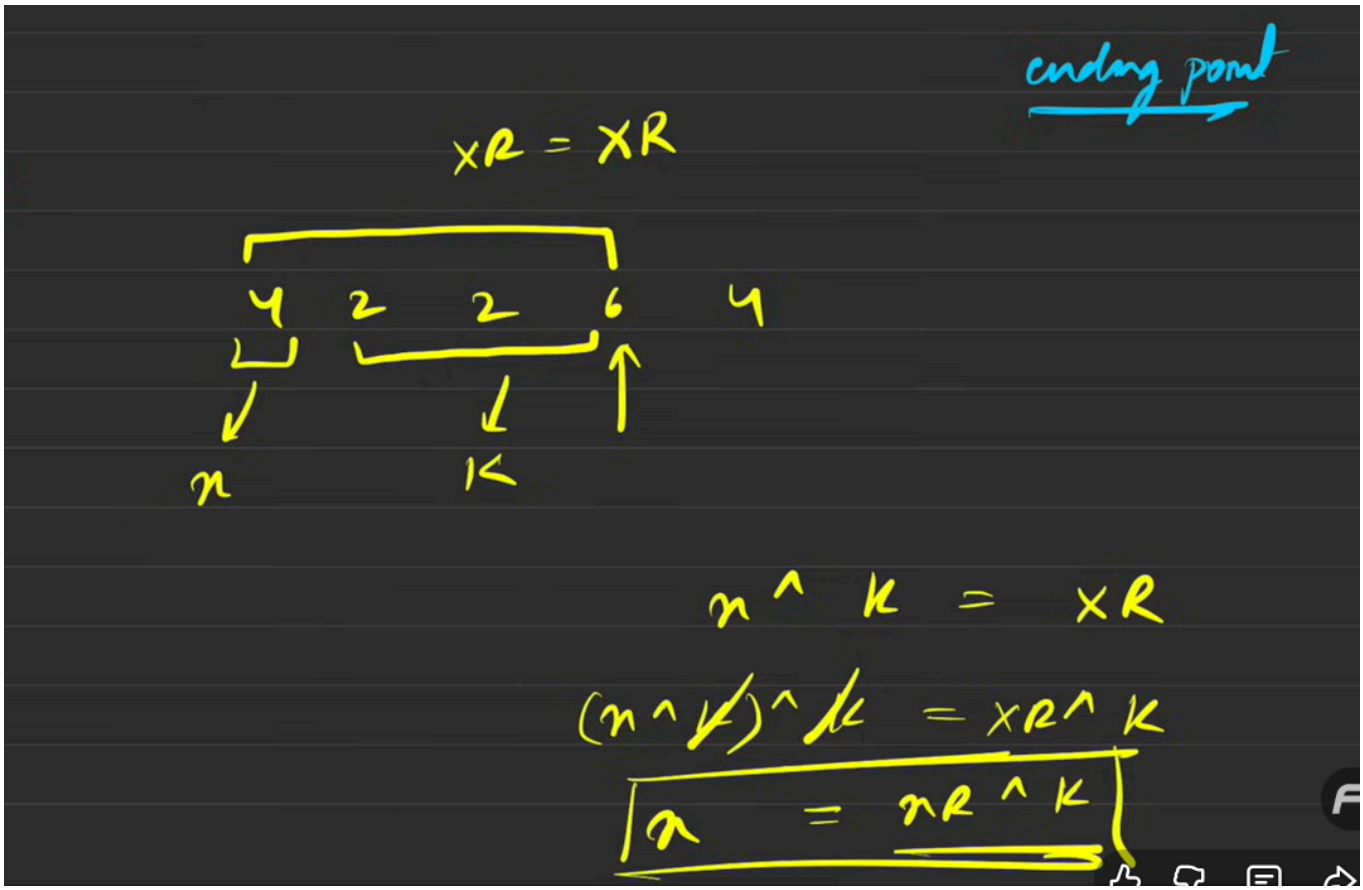
```
subXor = pref[r] ^ pref[l-1]     (l > 0)
subXor = pref[r]                 (l == 0)
```

Hume `subXor == K` chahiye ⇒
`pref[r] ^ pref[l-1] == K` ⇒
`pref[l-1] == pref[r] ^ K`

Matlab: jab hum right end `r` par hain aur `xr = pref[r]`, to jitni baar **`xr ^ K`** pehle dekha gaya hai (as a prefix), utni subarrays end at `r` with XOR `K`.

Isliye:

- ek map rakho: `freq[prefixXor]`
- start me `freq[0] = 1` (empty prefix)
- har element add karke `xr ^= a[i]`
    - `need = xr ^ K`
    - `cnt += freq[need]`
    - `freq[xr]++`




```
#include <bits/stdc++.h>
using namespace std;

int subarraysWithXorK(vector<int> a, int k) {
    int n = a.size(); //size of the given array.
    int xr = 0;
    map<int, int> mpp; //declaring the map.
    mpp[xr]++; //setting the value of 0.
    int cnt = 0;

    for (int i = 0; i < n; i++) {
        // prefix XOR till index i:
        xr = xr ^ a[i];

        //By formula: x = xr^k:
        int x = xr ^ k;

        // add the occurrence of xr^k
```

```
        // to the count:
        cnt += mpp[x];

        // Insert the prefix xor till index i
        // into the map:
        mpp[xr]++;
    }
    return cnt;
}

int main()
{
    vector<int> a = {4, 2, 2, 6, 4};
    int k = 6;
    int ans = subarraysWithXorK(a, k);
    cout << "The number of subarrays with XOR k is: "
         << ans << "\n";
    return 0;
}
```

Output: The number of subarrays with XOR k is: 4

Complexity Analysis

Time Complexity: O(N) or O(N*logN) depending on which map data structure we are using, where N = size of the array.
Reason: For example, if we are using an unordered_map data structure in C++ the time complexity will be O(N) but if we are using a map data structure, the time complexity will be O(N*logN). The least complexity will be O(N) as we are using a loop to traverse the array. Point to remember for unordered_map in the worst case, the searching time increases to O(N), and hence the overall time complexity increases to O(N2).

Space Complexity: O(N) as we are using a map data structure.

🔍 Quick Dry Run (same example)

`a = [4,2,2,6,4], K=6`
`freq = {0:1}`, `xr=0`, `cnt=0`

1. x=4 → `xr=4`, need=`4^6=2` → freq[2]=0 → cnt=0 → freq[4]=1
2. x=2 → `xr=6`, need=`6^6=0` → freq[0]=1 → cnt=1 → freq[6]=1
3. x=2 → `xr=4`, need=`4^6=2` → freq[2]=0 → cnt=1 → freq[4]=2

4. x=6 → `xr=2`, need=`2^6=4` → freq[4]=2 → cnt=3 → freq[2]=1
5. x=4 → `xr=6`, need=`6^6=0` → freq[0]=1 → cnt=4 → freq[6]=2
   Final `cnt=4` ✅

35. Merge Overlapping Sub-intervals

Intervals (start, end) diye hain. Hume **overlapping** intervals ko **merge** karke final non-overlapping list return karni hai.

Overlap rule (inclusive): Agar next interval ka `start <= current_end`, to overlap hai.

Example:
`[[1,3], [2,6], [8,10], [15,18]]` → `[[1,6], [8,10], [15,18]]`

🟠 Approach 1: Truly Brute Force (O(N²)) — without sorting

Idea: Har interval ke saath baaki sab ko compare karo; jo overlap kare, unhe merge karke ek block banao; visited mark karke skip karo.

Do intervals `[a,b]` aur `[c,d]` overlap karte hain agar:

```
c <= b   &&   a <= d

[1,3] aur [2,6] overlap karte hain

kyunki 2 <= 3
```

- **Steps:**
  1. `visited[n]=false`.
  2. For each `i` (not visited), `cur = [L,R] = intervals[i]`.

3. For each `j>i` (not visited): agar `[L,R]` aur `intervals[j]` overlap karein, to `L=min(L, start[j])`, `R=max(R, end[j])`, `visited[j]=true`, and restart inner scan (ya ek while loop se club kar lo).

4. `cur` ko answer me daal do.

Jb hm sort akrte hai to wo 1st element k according sort hota hai, lekin agar at any moment 1st element is same then It will sort according to 2nd element.

```
#include <bits/stdc++.h>
using namespace std;

bool overlap(const vector<int>& a, const vector<int>& b){
    return !(a[1] < b[0] || b[1] < a[0]); // inclusive overlap
}

vector<vector<int>> mergeBrute(vector<vector<int>> intervals){
    int n = intervals.size();
    vector<vector<int>> ans;
    vector<int> vis(n, 0);

    for(int i = 0; i < n; i++){
        if(vis[i]) continue;
        int L = intervals[i][0], R = intervals[i][1];
        vis[i] = 1;

        bool mergedAny = true;
        while(mergedAny){
            mergedAny = false;
            for(int j = 0; j < n; j++){
                if(vis[j]) continue;
                if(overlap({L,R}, intervals[j])){
                    L = min(L, intervals[j][0]);
                    R = max(R, intervals[j][1]);
                    vis[j] = 1;
                    mergedAny = true;
                }
            }
        }
        ans.push_back({L,R});
    }
    return ans;
}
```

Complexity Analysis
Time Complexity: O(N^2), for every interval we check all future intervals.
Space Complexity: ON), additonal space used to store the non-overlapping intervals.

🟢 Approach 2: Optimal — Sort + Single Pass (O(N log N))

Observation: Agar intervals ko start ke hisaab se sort kar doge, to overlaps **sirf pichle merged interval** se check karne padte hain. Isse sirf **ek pass** me merge ho jayega.

- **Steps:**
  1. `sort(intervals.begin(), intervals.end());`
  2. `merged` empty ho to push first.
  3. Har interval par:
     - Agar `interval.start > merged.back().end` → **no overlap** → push.
     - Warna **overlap** → `merged.back().end = max(merged.back().end, interval.end)`.
- **Time:** Sorting `O(N log N)` + merge pass `O(N)` ⇒ **O(N log N)**
- **Space:** If you return a new vector, **O(N)** (output size). In-place bhi kiya ja sakta hai, par interviews me clean return preferred hota hai.

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    // Function to merge overlapping intervals
    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        // Sort intervals based on starting time
        sort(intervals.begin(), intervals.end());

        // Vector to store final merged intervals
        vector<vector<int>> merged;

        // Traverse each interval
        for (auto interval : intervals) {
            // If merged is empty or current interval does not overlap
```

```
            if (merged.empty() || merged.back()[1] < interval[0]) {
                // Add current interval as a new non-overlapping block
                merged.push_back(interval);
            } else {
                // Overlapping: merge by extending the end time
                merged.back()[1] = max(
                    merged.back()[1],
                    interval[1]
                );
            }
        }

        return merged;
    }
};

int main() {
    Solution sol;
    vector<vector<int>> intervals = {
        {1, 3}, {2, 6}, {8, 10}, {15, 18}
    };

    vector<vector<int>> result = sol.merge(intervals);

    for (auto v : result) {
        cout << "[" << v[0] << "," << v[1] << "] ";
    }

    return 0;
}
```

Complexity Analysis

Time Complexity: O(N*logN) + O(N), we sort the entire array and then merge them in a single pass.

Space Complexity: ON), additonal space used to store the non-overlapping intervals.

36. Merge two Sorted Arrays Without Extra Space

Hume do **sorted arrays** diye gaye hain:

```
nums1 = [1, 3, 5, 0, 0, 0]
nums2 = [2, 4, 6]
```

`nums1` ke end me kuch extra `0`s diye gaye hain (placeholders),
taaki uske andar hi **nums2** ke elements merge kiye jaa sake.

👉 Goal: dono ko mila kar ek **sorted array** banana hai —
but **without using any extra array (O(1) space)**.

Expected Output:

```
[1, 2, 3, 4, 5, 6]
```

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    // Merges nums2 into nums1 in-place in sorted order.
    void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
        // i points to last valid element in nums1
        int i = m - 1;

        // j points to last element in nums2
        int j = n - 1;

        // k is the last index of nums1 (including 0 placeholders)
        int k = m + n - 1;

        // Fill nums1 from the end by comparing nums1[i] and nums2[j]
        while (i >= 0 && j >= 0) {
            // Place larger of the two at nums1[k]
            if (nums1[i] > nums2[j]) {
                nums1[k--] = nums1[i--];
            } else {
```

```
            nums1[k--] = nums2[j--];
        }
    }

    // If nums2 has remaining elements, copy them
    while (j >= 0) {
        nums1[k--] = nums2[j--];
    }

    // No need to copy remaining nums1 elements, as they are already in place
  }
};

int main() {
    vector<int> nums1 = {1, 3, 5, 0, 0, 0};
    vector<int> nums2 = {2, 4, 6};
    int m = 3, n = 3;

    Solution().merge(nums1, m, nums2, n);

    // Print merged array
    for (int num : nums1) cout << num << " ";
    return 0;
}
```

Complexity Analysis
Time Complexity: O(N+M), we traverse both the arrays exactly once.
Space Complexity: O(1), constant extra space is used to store pointers.

37. Find the repeating and missing numbers

Given an array a of size n,
which contains **numbers from 1 to n**,
but:

- ek number **repeats twice**,
- aur ek number **missing hai**.

Find both.

Input: a = [3, 1, 2, 5, 4, 6, 7, 5]
Output: Repeating = 5, Missing = 8

⚙️ Approach 1: Brute Force (O(N²))

💡 Idea:

1. For each number from 1 to n,
    - count how many times it appears in the array.
2. If any number appears twice → repeating.
3. If any number appears zero times → missing.

```
#include <bits/stdc++.h>
using namespace std;

vector<int> findMissingRepeatingNumbers(vector<int> a) {
    int n = a.size(); // size of the array
    int repeating = -1, missing = -1;

    //Find the repeating and missing number:
    for (int i = 1; i <= n; i++) {
        //Count the occurrences:
        int cnt = 0;
        for (int j = 0; j < n; j++) {
            if (a[j] == i) cnt++;
        }

        if (cnt == 2) repeating = i;
        else if (cnt == 0) missing = i;

        if (repeating != -1 && missing != -1)
            break;
    }
    return {repeating, missing};
}

int main()
{
    vector<int> a = {3, 1, 2, 5, 4, 6, 7, 5};
    vector<int> ans = findMissingRepeatingNumbers(a);
    cout << "The repeating and missing numbers are: {"
```

```
        << ans[0] << ", " << ans[1] << "}\n";
    return 0;
}
```

Output: The repeating and missing numbers are: {5, 8}

Complexity Analysis

Time Complexity: O(N2), where N = size of the given array.
Reason: Here, we are using nested loops to count occurrences of every element between 1 to N.

Space Complexity: O(1) as we are not using any extra space.

🟡 Approach 2 — Using Frequency Array (O(N), O(N))

🔹 Idea:

Ek **freq array** banao jisme har element ka count store karo.

- Agar freq[i] = 0 → missing
- Agar freq[i] = 2 → repeating

```
#include <bits/stdc++.h>
using namespace std;

vector<int> findMissingRepeatingNumbers(vector<int> a) {
    int n = a.size(); // size of the array
    int hash[n + 1] = {0}; // hash array

    //update the hash array:
    for (int i = 0; i < n; i++) {
        hash[a[i]]++;
    }

    //Find the repeating and missing number:
```

```
    int repeating = -1, missing = -1;
    for (int i = 1; i <= n; i++) {
        if (hash[i] == 2) repeating = i;
        else if (hash[i] == 0) missing = i;

        if (repeating != -1 && missing != -1)
            break;
    }
    return {repeating, missing};
}

int main()
{
    vector<int> a = {3, 1, 2, 5, 4, 6, 7, 5};
    vector<int> ans = findMissingRepeatingNumbers(a);
    cout << "The repeating and missing numbers are: {"
         << ans[0] << ", " << ans[1] << "}\n";
    return 0;
}
```

Output: The repeating and missing numbers are: {5, 8}

Complexity Analysis

Time Complexity: O(2N), where N = the size of the given array.
Reason: We are using two loops each running for N times. So, the time complexity will be O(2N).

Space Complexity: O(N) as we are using a hash array to solve this problem.

🟢 Approach 3 — Mathematical Formula (O(N), O(1))

🔹 Concept:

Let actual numbers = 1, 2, 3, ..., n
Let given numbers = array elements (with one missing, one repeating)

We know formulas:

```
Sum of 1..n = n(n+1)/2
Sum of squares of 1..n = n(n+1)(2n+1)/6
```

Now let:

- `S` = actual sum (1 + 2 + ... + n)
- `S2` = actual sum of squares
- `S_actual` = sum of given array
- `S2_actual` = sum of squares of given array

We have two equations:

```
(1)  S_actual - S = R - M
(2)  S2_actual - S2 = R² - M² = (R - M)(R + M)
```

From (1): `R - M = val1`
From (2): `R + M = val2 / val1`

Now:

```
R = (val1 + val2/val1) / 2
M = R - val1
```

```
#include <bits/stdc++.h>
using namespace std;

vector<int> findMissingRepeatingNumbers(vector<int> a) {
    long long n = a.size(); // size of the array

    // Find Sn and S2n:
    long long SN = (n * (n + 1)) / 2;
    long long S2N = (n * (n + 1) * (2 * n + 1)) / 6;

    // Calculate S and S2:
    long long S = 0, S2 = 0;
    for (int i = 0; i < n; i++) {
        S += a[i];
        S2 += (long long)a[i] * (long long)a[i];
    }

    //S-Sn = X-Y:
    long long val1 = S - SN;

    // S2-S2n = X^2-Y^2:
```

```
    long long val2 = S2 - S2N;

    //Find X+Y = (X^2-Y^2)/(X-Y):
    val2 = val2 / val1;

    //Find X and Y: X = ((X+Y)+(X-Y))/2 and Y = X-(X-Y),
    // Here, X-Y = val1 and X+Y = val2:
    long long x = (val1 + val2) / 2;
    long long y = x - val1;

    return {(int)x, (int)y};
}

int main()
{
    vector<int> a = {3, 1, 2, 5, 4, 6, 7, 5};
    vector<int> ans = findMissingRepeatingNumbers(a);
    cout << "The repeating and missing numbers are: {"
         << ans[0] << ", " << ans[1] << "}\n";
    return 0;
}
```

Output: The repeating and missing numbers are: {5, 8}

Complexity Analysis

Time Complexity: O(N), where N = the size of the given array.
Reason: We are using only one loop running for N times. So, the time complexity will be O(N).

Space Complexity: O(1) as we are not using any extra space to solve this problem.

🔵 Approach 4 — XOR Method (O(N), O(1)) (Can Ignore)

💡 Concept:

XOR of numbers cancels out equal numbers.

1️⃣ Take XOR of all elements in the array and all numbers from 1..n:

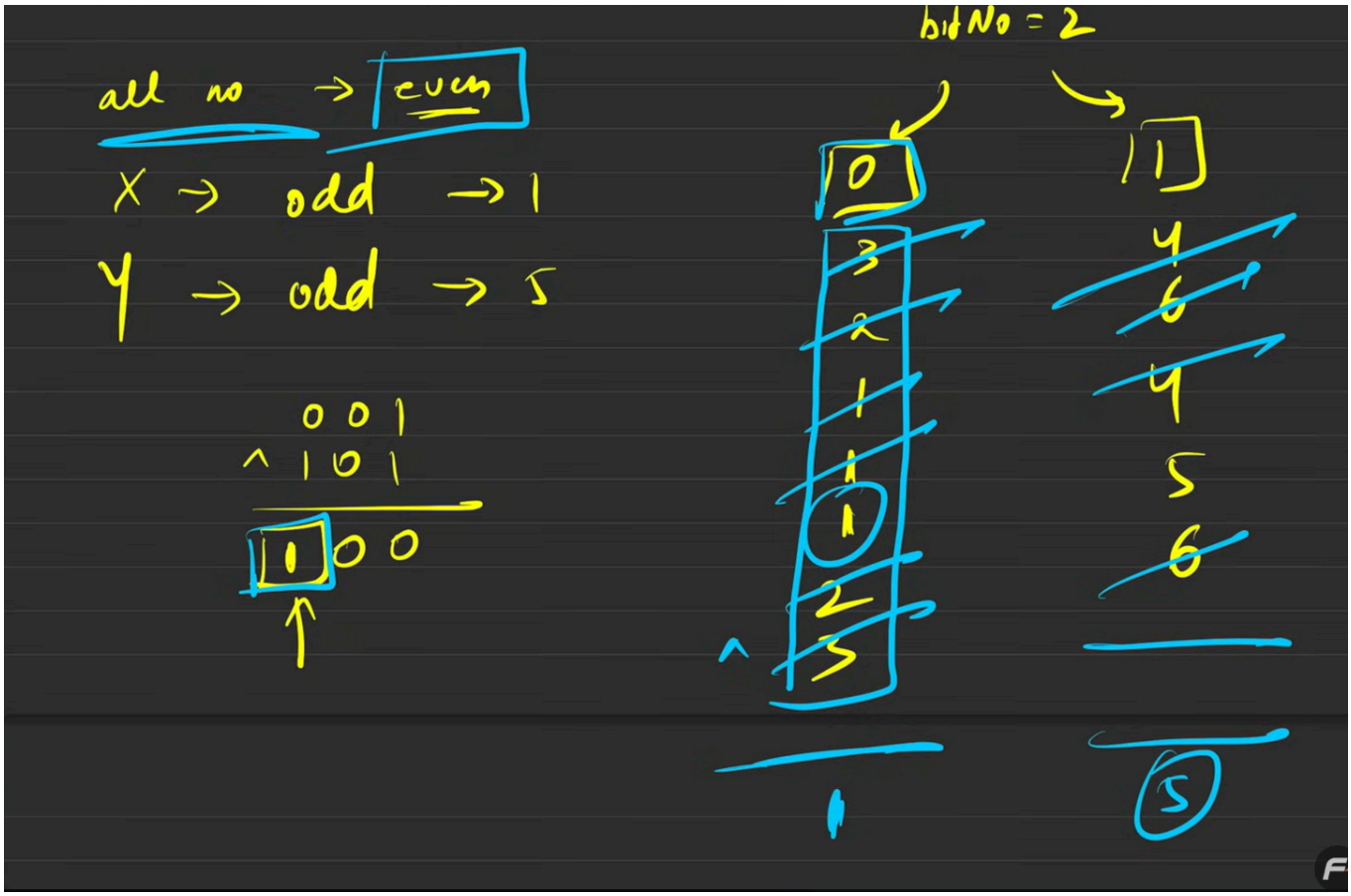
```
xorAll = (a[0] ^ a[1] ^ ... ^ a[n-1]) ^ (1 ^ 2 ^ ... ^ n)
```

The result = `R ^ M` (since all others cancel).

2️⃣ Find the rightmost set bit in `xorAll` (this helps separate R and M).

3️⃣ Divide numbers into two groups based on this bit and XOR separately.
That gives one as `R` and other as `M`.

Then, check which one actually repeats in the array.



🔹 Step 1: XOR Concept Recap

Important XOR rules:

```
a ^ a = 0       // same numbers cancel out
a ^ 0 = a       // XOR with zero changes nothing
a ^ b = b ^ a   // commutative
```

🔹 Step 2: First XOR of everything

We XOR **all array elements** and **all numbers from 1..n**.

```
xorAll = (3 ^ 1 ^ 2 ^ 5 ^ 4 ^ 6 ^ 7 ^ 5) ^ (1 ^ 2 ^ 3 ^ 4 ^ 5 ^ 6 ^ 7 ^ 8)
```

Why we do this?

Because **every number that appears exactly once**
(from both sides) will **cancel out** due to `a ^ a = 0`.

Only **two numbers** will remain:

- the one that **repeats** (`R`)
- the one that is **missing** (`M`)

So:

```
xorAll = R ^ M
```

👉 For our example:

```
xorAll = 5 ^ 8
```

🔹 Step 3: Problem — We only have XOR (5 ^ 8)

We know only that xorAll = 5 ^ 8,
but we can't directly tell which are 5 and 8 individually.

So we need to **separate 5 and 8 somehow**.

🔹 Step 4: Find a bit where 5 and 8 differ

Let's write their binary:

```
5 = 0101
8 = 1000
------------
xorAll = 1101
```

The `xorAll` bits that are `1` tell us **where 5 and 8 differ**.

If we pick **any one differing bit** (like the rightmost 1-bit),
then we can separate numbers based on that bit.

🔹 Step 5: Find rightmost set bit

We need to pick **one bit position** where these two differ,
so we choose the **rightmost set bit (1)** in xorAll.

For xorAll = 13 (binary 1101)

Rightmost set bit = `0001` (the last '1' bit from the right).

This bit position is **different in 5 and 8**:

```
5 = 0101  → last bit = 1
8 = 1000  → last bit = 0
```

So we can now divide numbers into **two groups**:

- Group 1 → those having this bit = 1
- Group 2 → those having this bit = 0

That's how we'll separate R and M.

🔹 Step 6: How to get that rightmost bit in code?

Line:

```
int rightmostSetBit = xorAll & (~(xorAll - 1));
```

Let's decode this 🔍

Say:

```
xorAll = 1101 (binary)
xorAll - 1 = 1100
~(xorAll - 1) = 0011
```

Now:

```
xorAll & (~(xorAll - 1))
= 1101
& 0011
= 0001
```

✅ It gives **only the rightmost set bit**.
(i.e., the bit position where the least significant '1' occurs.)

So now `rightmostSetBit = 0001` in binary (means bit position 1).

🔹 Step 7: Divide into two groups using this bit

Now traverse again over:

1. all elements in array, and
2. all numbers from 1 to n.

If that bit (0001) is **set**, put in Group X.
If not set, put in Group Y.

Then XOR all numbers within each group.

Because in each group, same numbers will again cancel out,
and only one unique number (either R or M) will remain.

Example (continue with our array):

Rightmost bit mask = `0001`

Number	Bit (0001)?	Group
3 (0011)	yes	X
1 (0001)	yes	X
2 (0010)	no	Y
5 (0101)	yes	X
4 (0100)	no	Y
6 (0110)	no	Y
7 (0111)	yes	X
5 (0101)	yes	X
1..8	similarly divided	both sides cancel except one in each group

After XORing within groups:

Group X → gives one number (say 5)
Group Y → gives the other (say 8)

Now we have **R and M** but don't know which is which.
We just check the array once:

- If number appears twice → repeating
- Otherwise → missing

```
vector<int> findMissingRepeatingNumbers(vector<int> a) {
    int n = a.size();
    int xorAll = 0;

    // Step 1: XOR of all array elements and numbers 1..n
    for (int i = 0; i < n; i++)
        xorAll ^= a[i];
    for (int i = 1; i <= n; i++)
        xorAll ^= i;

    // Step 2: xorAll = R ^ M
    int rightmostSetBit = xorAll & (~(xorAll - 1));

    int x = 0, y = 0; // two buckets

    // Step 3: Divide into groups using the rightmost set bit
    for (int i = 0; i < n; i++) {
        if (a[i] & rightmostSetBit)
            x ^= a[i];
        else
            y ^= a[i];
    }
    for (int i = 1; i <= n; i++) {
        if (i & rightmostSetBit)
            x ^= i;
        else
            y ^= i;
    }

    // Step 4: x and y are R and M (check which is repeating)
    int count = 0;
    for (int num : a)
        if (num == x) count++;

    if (count == 2)
        return {x, y}; // x is repeating
    else
        return {y, x}; // y is repeating
}
```

Complexity Analysis

Time Complexity: O(N), where N = the size of the given array.
Reason: We are just using some loops running for N times. So, the time complexity will be approximately O(N).

Space Complexity: O(1) as we are not using any extra space to solve this problem.

38. Count inversions in an array

Inversion: pair `(i, j)` jahan `i < j` aur `a[i] > a[j]`.

Example: `a = [5,4,3,2,1]`
All pairs inverted → count = `10`.

🟠 Approach 1: Brute Force — O(N²), O(1)

Har `(i, j)` (i < j) pair check karo:

```
#include <bits/stdc++.h>
using namespace std;

int numberOfInversions(vector<int>&a, int n) {

    // Count the number of pairs:
    int cnt = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] > a[j]) cnt++;
        }
```

```
    }
    return cnt;
}

int main()
{
    vector<int> a = {5, 4, 3, 2, 1};
    int n = 5;
    int cnt = numberOfInversions(a, n);
    cout << "The number of inversions is: "
         << cnt << endl;
    return 0;
}
```

Output: The number of inversions is: 10

Complexity Analysis

Time Complexity: O(N2), where N = size of the given array.
Reason: We are using nested loops here and those two loops roughly run for N times.

Space Complexity: O(1) as we are not using any extra space to solve this problem.

🟢 Approach 2: Merge Sort se Count — O(N log N), O(N)

Core idea (intuition)

- Agar dono halves individually sorted hain:
  - Left: `L = a[low..mid]`
  - Right: `R = a[mid+1..high]`
- Merge karte waqt jab `R[right] < L[left]` milta hai, to **Left me `left..mid` ke saare elements** `R[right]` se bade hote hain (kyunki left half sorted hoti hai).
  Isliye **ek shot me `(mid - left + 1)` inversions** add kar dete hain.

```
#include <bits/stdc++.h>
using namespace std;

long long mergeAndCount(vector<int>& arr, int low, int mid, int high){
    vector<int> temp;
```

```
    int i = low, j = mid+1;
    long long cnt = 0;

    while(i <= mid && j <= high){
        if(arr[i] <= arr[j]){
            temp.push_back(arr[i++]);
        } else {
            temp.push_back(arr[j++]);
            // arr[i]..arr[mid] sab > arr[j-1]
            cnt += (mid - i + 1);
        }
    }
    while(i <= mid) temp.push_back(arr[i++]);
    while(j <= high) temp.push_back(arr[j++]);

    for(int k=low; k<=high; k++) arr[k] = temp[k - low];
    return cnt;
}

long long mergeSortCount(vector<int>& arr, int low, int high){
    if(low >= high) return 0;
    int mid = (low + high) / 2;
    long long cnt = 0;
    cnt += mergeSortCount(arr, low, mid);
    cnt += mergeSortCount(arr, mid+1, high);
    cnt += mergeAndCount(arr, low, mid, high);
    return cnt;
}

long long numberOfInversions(vector<int>& a, int n){
    // NOTE: ye function array ko sort bhi kar deta hai (side-effect).
    // Agar original order preserve karna ho to copy pe kaam karo.
    return mergeSortCount(a, 0, n-1);
}

int main(){
    vector<int> a = {5,4,3,2,1};
    cout << "The number of inversions is: "
         << numberOfInversions(a, (int)a.size()) << "\n";
}
```

Complexity Analysis

Time Complexity: O(N*logN), where N = size of the given array.
Reason: We are not changing the merge sort algorithm except by adding a variable to it. So, the time complexity is as same as the merge sort.

Space Complexity: O(N), as in the merge sort We use a temporary array to store elements in sorted order.

39. Count Reverse Pairs

pairs `(i, j)` with `i < j` **and** `a[i] > 2 * a[j]`.
Example: `a = [4,1,2,3,1]` → answer **3**

🟠 Brute Force (O(N²), O(1))

Har `(i, j)` check karo:

```
#include <bits/stdc++.h>
using namespace std;

int countPairs(vector<int>&a, int n) {

    // Count the number of pairs:
    int cnt = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] > 2 * a[j]) cnt++;
        }
    }
    return cnt;
}

int team(vector <int> & skill, int n) {
```

```
    return countPairs(skill, n);
}

int main()
{
    vector<int> a = {4, 1, 2, 3, 1};
    int n = 5;
    int cnt = team(a, n);
    cout << "The number of reverse pair is: "
         << cnt << endl;
    return 0;
}
```

Output: The number of reverse pair is: 3

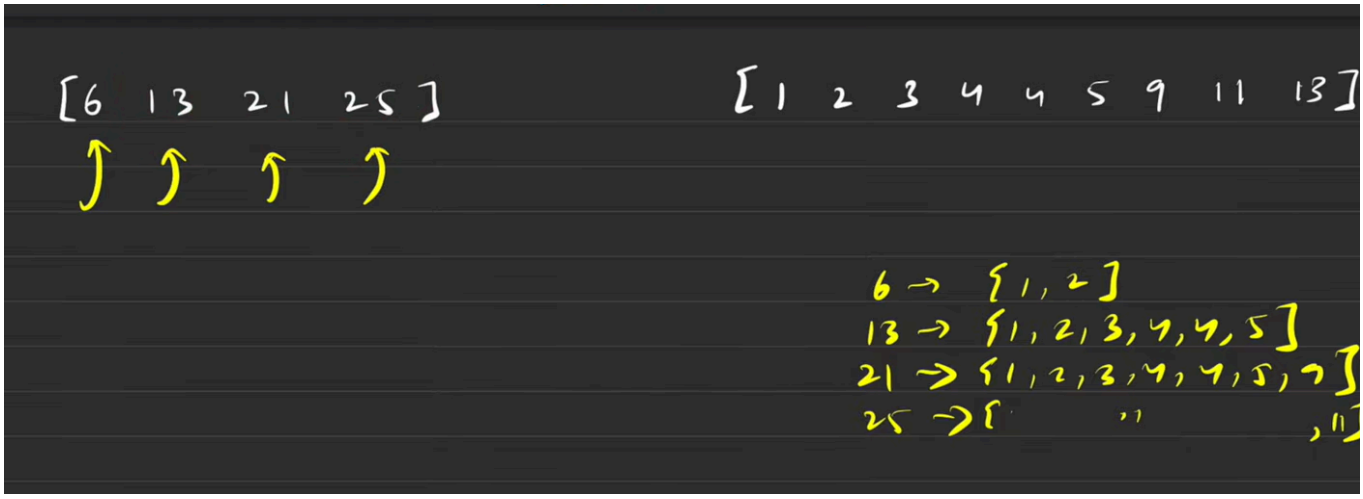
Time Complexity: O(N2), where N = size of the given array.
Reason: We are using nested loops here and those two loops roughly run for N times.

Space Complexity: O(1) as we are not using any extra space to solve this problem.

🟢 Optimal: Merge Sort + Two-Pointer Count (O(N log N), O(N))

Intuition

- Merge sort me halves **sorted** hote hain.
- Left half `L = a[low..mid]`, right half `R = a[mid+1..high]`.
- For each `i` in `L`, jitne `R[right]` aise hain ki `L[i] > 2*R[right]`, **wo sab valid**.
  Kyunki `R` sorted hai, right-pointer ko **sirf aage** badhata hai (reset nahi).




```
#include <bits/stdc++.h>
using namespace std;

void merge(vector<int> &arr, int low, int mid, int high) {
    vector<int> temp; // temporary array
    int left = low;      // starting index of left half of arr
    int right = mid + 1;   // starting index of right half of arr

    //storing elements in the temporary array in a sorted manner//

    while (left <= mid && right <= high) {
        if (arr[left] <= arr[right]) {
            temp.push_back(arr[left]);
            left++;
        }
        else {
            temp.push_back(arr[right]);
            right++;
        }
    }

    // if elements on the left half are still left //

    while (left <= mid) {
        temp.push_back(arr[left]);
        left++;
    }

    //  if elements on the right half are still left //
    while (right <= high) {
```

```
        temp.push_back(arr[right]);
        right++;
    }

    // transfering all elements from temporary to arr //
    for (int i = low; i <= high; i++) {
        arr[i] = temp[i - low];
    }
}

int countPairs(vector<int> &arr, int low, int mid, int high) {
    int right = mid + 1;
    int cnt = 0;
    for (int i = low; i <= mid; i++) {
        while (right <= high && arr[i] > 2 * arr[right]) right++;
        cnt += (right - (mid + 1));
    }
    return cnt;
}

int mergeSort(vector<int> &arr, int low, int high) {
    int cnt = 0;
    if (low >= high) return cnt;
    int mid = (low + high) / 2 ;
    cnt += mergeSort(arr, low, mid);  // left half
    cnt += mergeSort(arr, mid + 1, high); // right half
    cnt += countPairs(arr, low, mid, high); //Modification
    merge(arr, low, mid, high);  // merging sorted halves
    return cnt;
}

int team(vector <int> & skill, int n)
{
    return mergeSort(skill, 0, n - 1);
}

int main()
{
    vector<int> a = {4, 1, 2, 3, 1};
    int n = 5;
    int cnt = team(a, n);
    cout << "The number of reverse pair is: "
         << cnt << endl;
    return 0;
```

}

Output: The number of reverse pair is: 3

Time Complexity: O(2N*logN), where N = size of the given array.
Reason: Inside the mergeSort() we call merge() and countPairs() except mergeSort() itself. Now, inside the function countPairs(), though we are running a nested loop, we are actually iterating the left half once and the right half once in total. That is why, the time complexity is O(N). And the merge() function also takes O(N). The mergeSort() takes O(logN) time complexity. Therefore, the overall time complexity will be O(logN * (N+N)) = O(2N*logN).

Space Complexity: O(N), as in the merge sort We use a temporary array to store elements in sorted order.

40. Maximum Product Subarray in an Array

1) Brute Force — O(N²), O(1)

Idea: har `i` se start karke product carry-forward karo, har `j` par update.

- Kyun? Product recompute karne ke bajay, last product ko `* nums[j]` karke aage badh jao.
- Negatives/zeros handle ho jaate hain kyunki hum sab subarrays try kar rahe.

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    // This function finds the maximum product
    // of any contiguous subarray using brute force
    int maxProduct(vector<int>& nums) {
        // Initialize the answer with the first element
        int maxProd = nums[0];

        // Outer loop picks the starting index
        for (int i = 0; i < nums.size(); i++) {
            // Initialize current product to 1
```

```
            int prod = 1;

            // Inner loop picks the ending index
            for (int j = i; j < nums.size(); j++) {
                // Multiply current number to product
                prod *= nums[j];

                // Update maximum product if needed
                maxProd = max(maxProd, prod);
            }
        }

        // Return the result
        return maxProd;
    }
};

int main() {
    // Sample input
    vector<int> nums = {2, 3, -2, 4};

    // Create Solution object
    Solution sol;

    // Print the result
    cout << sol.maxProduct(nums);

    return 0;
}
```

Complexity Analysis
Time Complexity: O(N^2), we check the product of all possible subarrays using two nested loops.
Space Complexity: O(1), No extra space is used.

2) Prefix–Suffix Trick — O(N), O(1)

Observation:

- **Zero** product ko **break-point** samjho: zero aate hi koi bhi running product 0 ho jaata hai; zero ke baad naya subarray start maan lo.
- **Negative numbers**: odd count of negatives → overall product negative; par agar tum **aage se (prefix)** aur **peeche se (suffix)** dono taraf se multiply karo, to “odd negative ko cut” ho sakta hai

(jaise left se ek negative chhodke, right se ek chhodke).

- Isliye hum **left→right** aur **right→left** ek hi pass me products nikaalte hain, aur beech me zero milte hi running product ko **1** reset kar dete hain. Har step par `ans = max(ans, max(prefixProd, suffixProd))`.

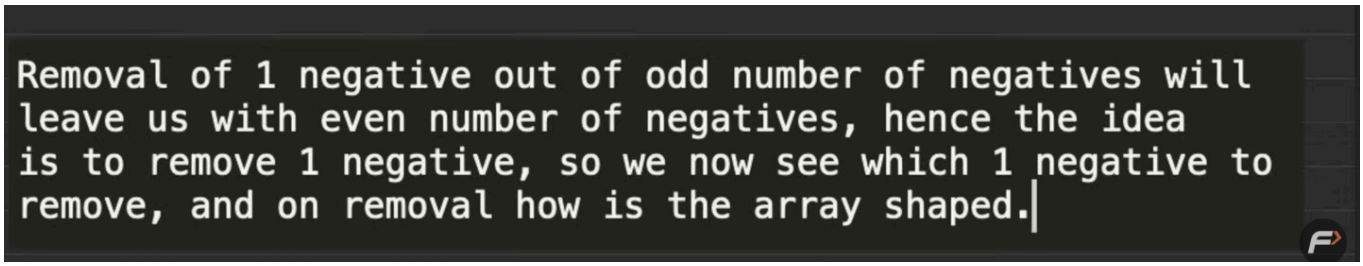
Dry Run (arr = `[2, 3, -2, 4]`):

- Prefix: 2 → 6 → -12 → -48 → max so far = 6 (baad me answer 6 banega)
- Suffix: 4 → -8 → -24 → -48 → max so far = 6
  Final = **6** (`[2,3]`).

Zero Case (arr = `[0, -2, -3, 0, -2, -4]`):

- Prefix me zero pe reset (pre=1), Suffix me bhi reset.
- Left/right pass milke best chunk pick ho jaata.

Tip: `ans` ko `INT_MIN` rakhna sahi; values bada range ho sakti hain to **`long long`** prefix/suffix use karna safe hota hai.




```
#include <bits/stdc++.h>
using namespace std;

// This function returns the maximum product subarray
// using prefix and suffix traversal
class Solution {
public:
    int maxProductSubArray(vector<int>& arr) {
        // Store size of array
```

```
        int n = arr.size();

        // Initialize prefix and suffix product
        int pre = 1, suff = 1;

        // Initialize answer to negative infinity
        int ans = INT_MIN;

        // Traverse from both left and right
        for (int i = 0; i < n; i++) {
            // Reset prefix if zero
            if (pre == 0) pre = 1;

            // Reset suffix if zero
            if (suff == 0) suff = 1;

            // Multiply prefix with current element from front
            pre *= arr[i];

            // Multiply suffix with current element from back
            suff *= arr[n - i - 1];

            // Update the maximum of all products seen so far
            ans = max(ans, max(pre, suff));
        }

        // Return the final answer
        return ans;
    }
};

int main() {
    // Sample input
    vector<int> arr = {2, 3, -2, 4};

    // Create object of solution
    Solution obj;

    // Call the function and print the result
    cout << obj.maxProductSubArray(arr) << endl;

    return 0;
}
```

Complexity Analysis

Time Complexity: O(N), every element of array is visited once.
Space Complexity: O(1), constant number of variables are used.

3) Optimal DP (Kadane-style for Product) — O(N), O(1) (IGNORE)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    // This function returns the maximum product
    // of any contiguous subarray using optimized approach
    int maxProduct(vector<int>& nums) {
        // Initialize answer, max and min product as first element
        int res = nums[0];
        int maxProd = nums[0];
        int minProd = nums[0];

        // Traverse from second element
        for (int i = 1; i < nums.size(); i++) {
            // Store current number
            int curr = nums[i];

            // If current number is negative, swap max and min
            if (curr < 0) swap(maxProd, minProd);

            // Update max and min product ending at current index
            maxProd = max(curr, maxProd * curr);
            minProd = min(curr, minProd * curr);

            // Update global result
            res = max(res, maxProd);
        }

        // Return the result
        return res;
    }
};

int main() {
    vector<int> nums = {2, 3, -2, 4};
```

```
    Solution sol;
    cout << sol.maxProduct(nums);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N), every element of array is visited once.

Space Complexity: O(1) , only constant variables are used.